



FM의 Fine Tuning과 RAG 활용

2026

- **NLP란?**

- 자연어처리(NLP)란 인간 언어를 컴퓨터가 이해·생성·처리하도록 하는 기술
- 제조 현장 적용: 불량 보고서, 점검 기록, SOP 문서 등 비정형 텍스트 데이터 처리
- 주요 task: 분류, 개체명 인식(NER), 요약, 생성, 질의응답(QA)
- LLM 등장 이후 여러 task를 단일 모델로 통합 수행하는 흐름으로 전환

1. NLP와 LLM

- 자연어 처리 = 언어에 대한 분석 ?
 - Noam Chomsky, 언어는 인간 정신 세계의 반영, 의미와 어휘의 역할 강조
 - 형태소: 언어를 분석하기 위한 기본 단위, 의미를 가지는 요소로서 더 이상 분석할 수 없는 가장 작은 문법 단위, 단순어의 어근, 어미, 조사, 접두사, 접미사 등
 - 음소(phoneme) < 형태소 (morpheme) < 단어(word) < 문장(sentence) < 텍스트(text) < 코퍼스(corpus)
 - Corpus
 - 언어 데이터를 대량으로 수집하고 언어 현상의 조사, 통계처리 등을 목적으로 수집된 언어 데이터

1. NLP와 LLM

- 자연어 처리
 - 형태소 분석 (morphological analysis)
 - 입력된 문자열을 분석하여 형태소라는 자연언어 분석을 위한 기본 단위로 나누는 것 (형태소 결합 규칙을 역으로 적용)
 - 예) 데이터는 → '데이터 (명사) + 는 (조사)'
 - 구문 분석 (syntactic analysis)
 - 형태소들이 결합하여 문장이나 구절을 만드는 구문 규칙에 따라서 문장 내에서 각 형태소들이 가지는 역할 (주어, 목적어)을 분석하는 것
 - 예) 통계학자는 데이터를 분석한다. (통계학자: 주어, 분석한다: 서술어)

1. NLP와 LLM

- 자연어 처리
 - 의미 분석 (semantic analysis)
 - 구문 분석의 결과를 해석하여 문장이 가지는 의미 (형태소의 의미)를 분석하는 것
 - 예) 말은 말을 못한다.
 - 실용 분석 (pragmatic analysis)
 - 문장이 실세계와 가지는 연관 관계를 분석하는 것
 - 예) 놀고있네
 - 잘 놀고 있다 또는 질책의 의미
 - 실세계 지식이나 상식 등을 토대로 화자와 청자의 대화 의도를 분석

- 한글에서의 형태소 분석

- 교착어 특징

- 한글 NLP에서 조사의 분리: 한국어는 어절이 독립적인 단어로 구성되는 것이 아니며, 조사 등이 붙어있는 상태. 형태소 분석을 위해서는 어절의 분리가 필요
 - 영어NLP: 독립적인 단어가 띄어쓰기 단위로 토큰화

- 형태소 분석(=lexical analysis): 용언의 불규칙활용 등의 단어에 대해서 원형 복원해야 함

- 한국어 토큰화의 형태소(morpheme)
 - 자립 형태소 : 접사, 어미, 조사와 상관없이 자립하여 사용할 수 있는 형태소. 그 자체로 단어. 체언(명사, 대명사, 수사), 수식언(관형사, 부사), 감탄사 등.
 - 의존 형태소 : 다른 형태소와 결합하여 사용되는 형태소. 접사, 어미, 조사, 어간 등.

1. NLP와 LLM

- 형태소 분석 예시

- 정통이가 통계책을 읽었다
 - 자립 형태소 : 정통이, 통계책
 - 의존 형태소 : -가, -을, 읽-, -었, -다
- 한글 NLP에서는 어절 토큰화가 아니라 형태소 토큰화를 수행해야 함

- 띄어쓰기의 문제

- 한글: 띄어쓰기가 지켜지지 않아도 이해가능한 언어
- 띄어쓰기가 없던 한국어에 띄어쓰기가 보편화된 것도 근대(1933년, 한글맞춤법통일안)에 시작
- 한글 띄어쓰기의 어려움
-

1. NLP와 LLM

- 토큰

- 토큰(token): 모델이 처리하는 최소 단위 — 문자/서브워드/단어 단위로 분할
- 토큰화 과정: 텍스트 → 서브워드 분리 → token ID 매핑 → 모델 입력
- 토큰 수 = context window 사용량과 직결 (API 비용, 처리 속도에 영향)
- 예: 제조 도메인 텍스트(설비명·전문용어)로 토큰화

1. NLP와 LLM

- **N-gram이란**

- 문장을 개별 단어 별 토큰화 할 경우, 문맥적인 의미는 손실
- N-gram은 연속된 n개의 단어를 하나의 토큰화 단위로 분리하는 것
- n개 단어 크기 윈도우를 만들어 문장의 처음부터 오른쪽으로 움직이면서 토큰화 수행
- 예 'Here is a dog'
 - 1-gram (단어 단위)
 - Here, is, a, dog
 - 1-gram (with 3-sized window)
 - "Her", "ere", "re_", "e_i", "_is", "is_", "s_a", "_a_", "a_d", "_do", "dog"
 - 2-gram(단어 단위)
 - "Here is", "is a", "a dog"

- **Byte Pair Encoding**

- BPE: 빈번하게 등장하는 문자/서브워드 쌍을 반복적으로 병합해 vocabulary를 구성하는 토큰화 알고리즘
- OOV(Out-of-Vocabulary) 문제 완화 — 신조어·전문용어도 서브워드 단위로 분해 가능
- 한국어 적용 한계: 형태소 단위와 불일치 → 의미 단위가 깨질 수 있음 (예: "베어링이" 분리 양상)
- GPT 계열은 BPE, BERT 계열은 WordPiece, 최근 모델은 SentencePiece/Unigram 변형 사용

1. NLP와 LLM

- **Byte Pair Encoding**

- BPE란?

- 1994년에 제안된 데이터 압축 알고리즘.
 - Unknown Token(UNK) 처리 가능: 모든 Term을 미리 알 수 없음, Out Of Vocabulary (OOV) 문제

- 예: aaabdaaabc에 대한 BPE

- 연속하여 많이 등장하는 문자(Byte로 표현)의 Pair를 찾아서 하나의 글자로 변환하여 압축
 - 위의 문자열 중 자주 등장하는 Byte Pair: aa이며 다른 Byte인 Z로 변환
 - » ZabdZabac / Z=aa
 - 동일하게 ab를 Y로 변환
 - » ZYdZYac / Y=ab, Z=aa
 - 동일하게 ZY를 X로 변환
 - » XdXac / X=ZY, Y=ab, Z=aa
 - 더 이상 변환할 Byte Pair가 없으면 종료

1. NLP와 LLM

- **자연어 처리에서의 Byte Pair Encoding**

- Subword Segmentation: Byte 단위에서 Vocabulary를 생성(문자로 나눈 후, 많이 등장하는 문자들을 묶어감)
- 예: 데이터 내 단어 현황
 - low : 5, lower : 2, newest : 6, widest : 3
- 위 단어 집합 외 Lowest가 등장하면 UNK

- **BPE 적용**

- l o w : 5, l o w e r : 2, n e w e s t : 6, w i d e s t : 3
- Vocabulary: l, o, w, e, r, n, s, t, i, d
- BPE 적용
 - 횟수는 지정 가능: 10회
 - 빈번한 한 문자(Unigram)의 쌍을 하나의 Unigram으로 치환
- ✓ 1st: e, s를 es로 병합
- ✓ 2nd : es와 t를 est로 병합
- ✓

<#>

1. NLP와 LLM

- **BPE 적용**

- 최종BPE 결과: low : 5, low e r : 2, newest : 6, widest : 3
 - Vocab: l, o, w, e, r, n, s, t, i, d, es, est, lo, low, ne, new, newest, wi, wid, widest
- 신규 단어 인식
 - lowest는 OOV?
 - Lowest->l o w e s t로 나눈 후 Vocab과 병합이력을 확인
 - 병합 과정에서 병합이력을 바탕으로 먼저 빈번했던 Unigram 쌍을 우선 병합
 - 1st: lo w es t
 - 2nd : low est
 - 이후 lowest는 vocab에 없으니 중지
 - 새로운 Term인 Lowest는 low와 est로 인코딩

1. NLP와 LLM

- BPE 적용
 - 신규 단어 인식 예 2
 - pest는 OOV?
 - pest->p e s t로 나눈 후 Vocab과 병합이력을 확인
 - 병합 과정에서 병합이력을 바탕으로 먼저 빈번했던 Unigram 쌍을 우선 병합
 - p는 vocab에 없으므로 <UNK>로 표현
 - 1st: <UNK> es t
 - 2nd : <UNK> est
 - 새로운 Term인 pest는 미등록 토큰과 est로 인코딩
 - 일반적으로 알파벳 등 개별 문자들은 BPE 어휘 집합을 구축할 때 초기 사전에 들어가므로 미등록 토큰이 발생하는 경우는 많지 않음
 - BPE는 어휘 집합 크기를 합리적으로 유지, 신조어 등에 대해 유의미한 분절을 수행

1. NLP와 LLM

- **LLM**

- LLM: 대규모 텍스트로 사전학습된 언어모델, 파라미터 규모 수십억~수천억 단위
- 동작 원리: next token prediction — 다음 토큰을 순차적으로 예측해 문장 생성
- 파라미터 규모와 성능: scale 증가 시 emergent ability(복잡한 추론 능력) 등장
- 온프레미스(Ollama 등) vs 클라우드 API 비교 — 제조 현장은 보안상 온프레미스 선호 경향

1. NLP와 LLM

- Scaling Laws (Open AI, 2020)
 - 컴퓨팅 리소스, 데이터, 모형 크기를 늘릴 수록 성능 개선
 - 새로운 능력이 생겨남 (Emergent Abilities)

Scaling Laws for Neural Language Models			
Jared Kaplan *		Sam McCandlish *	
Johns Hopkins University, OpenAI		OpenAI	
jaredk@jhu.edu		sam@openai.com	
Tom Henighan	Tom B. Brown	Benjamin Chess	Rewon Child
OpenAI	OpenAI	OpenAI	OpenAI
henighan@openai.com	tom@openai.com	bchess@openai.com	rewon@openai.com
Scott Gray	Alec Radford	Jeffrey Wu	Dario Amodei
OpenAI	OpenAI	OpenAI	OpenAI
scott@openai.com	alec@openai.com	jeffwu@openai.com	damodei@openai.com

Abstract

We study empirical scaling laws for language model performance on the cross-entropy loss. The loss scales as a power-law with model size, dataset size, and the amount of compute used for training, with some trends spanning more than seven orders of magnitude. Other architectural details such as network width or depth have minimal effects within a wide range. Simple equations govern the dependence of overfitting on model/dataset size and the dependence of training speed on model size. These relationships allow us to determine the optimal allocation of a fixed compute budget. Larger models are significantly more sample-efficient, such that optimally compute-efficient training involves training very large models on a relatively modest amount of data and stopping significantly before convergence.

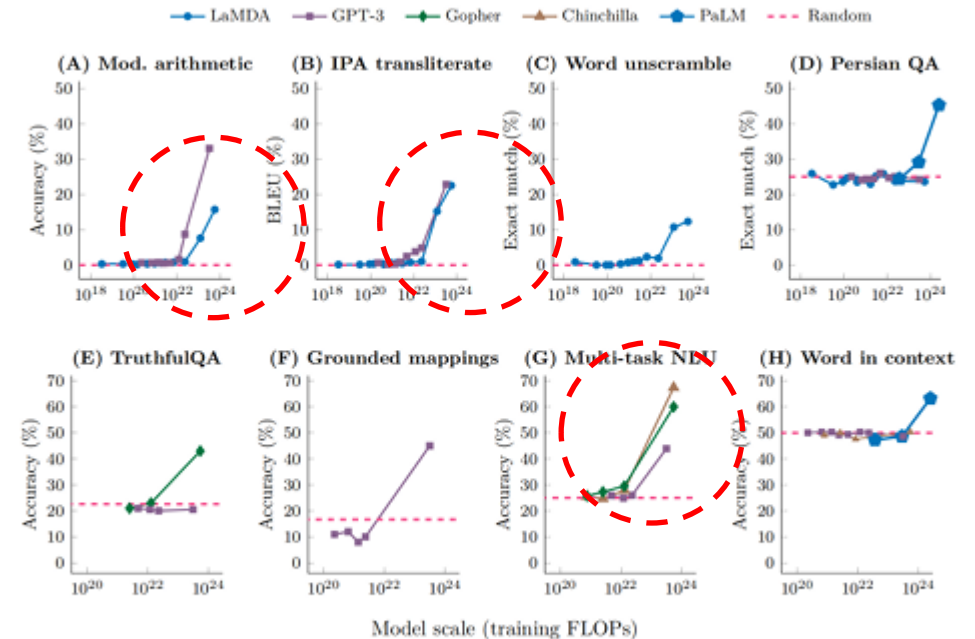
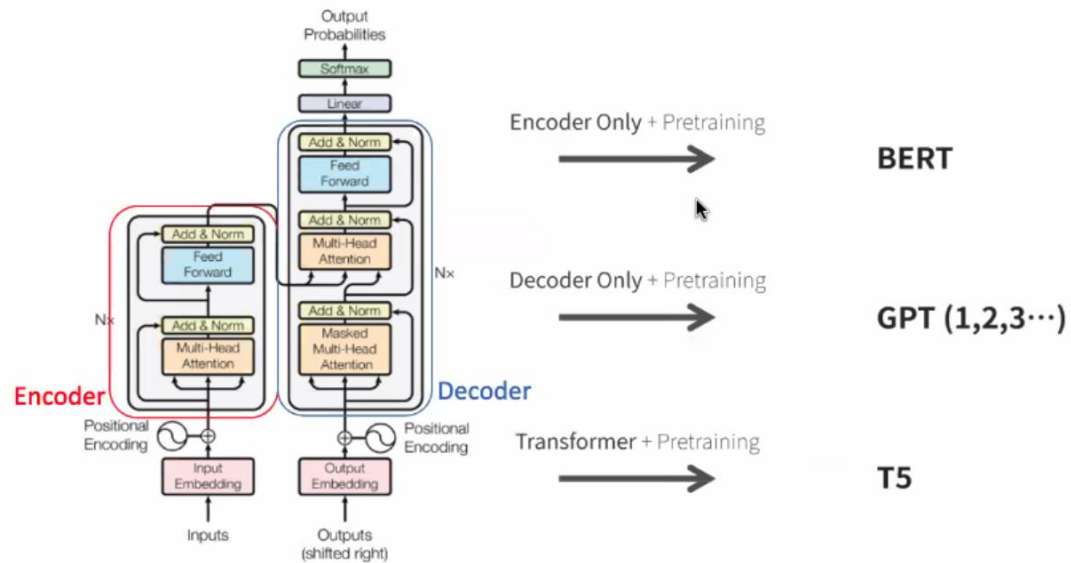


Figure 1: Emergent abilities of large language models. Model families display *sharp* and *unpredictable* increases in performance at specific tasks as scale increases. Source: Fig. 2 from [33].

1. NLP와 LLM

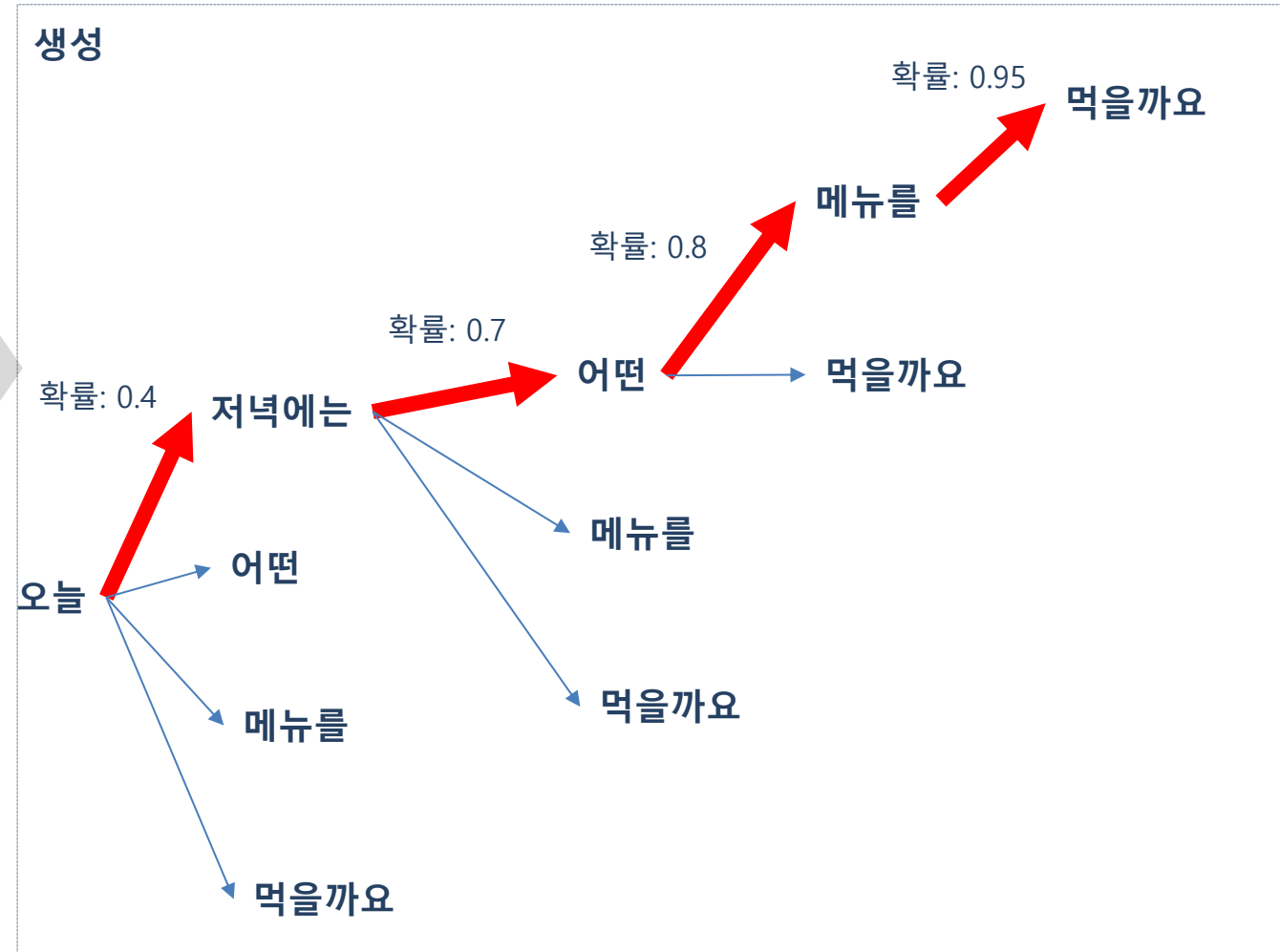
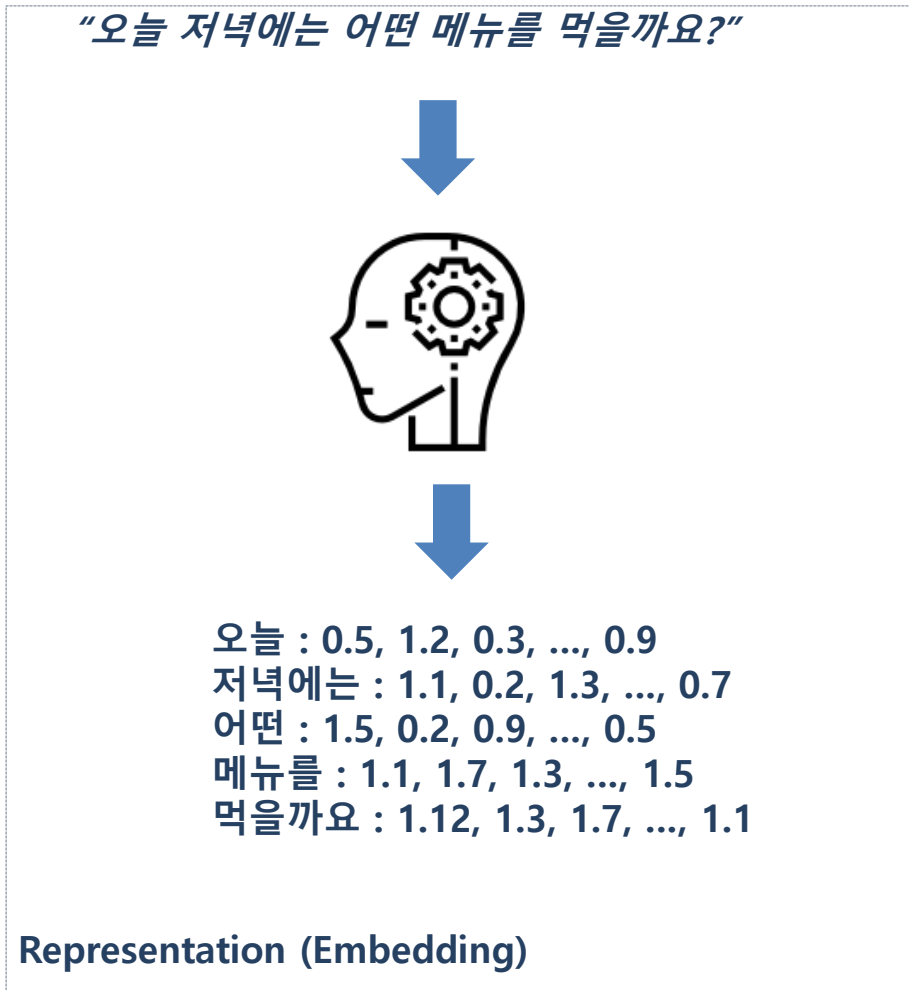
- 간단하게 이해하는 LLM

- 언어모형: 주어진 문장을 구성하는 단어에 대한 확률을 모델링
- 주어진 문장/단어의 다음 단어를 예측, 문장 내 Masking한 단어를 예측
- 대용량 텍스트를 학습한 모형이 좋은 Representation(Embedding)을 보유



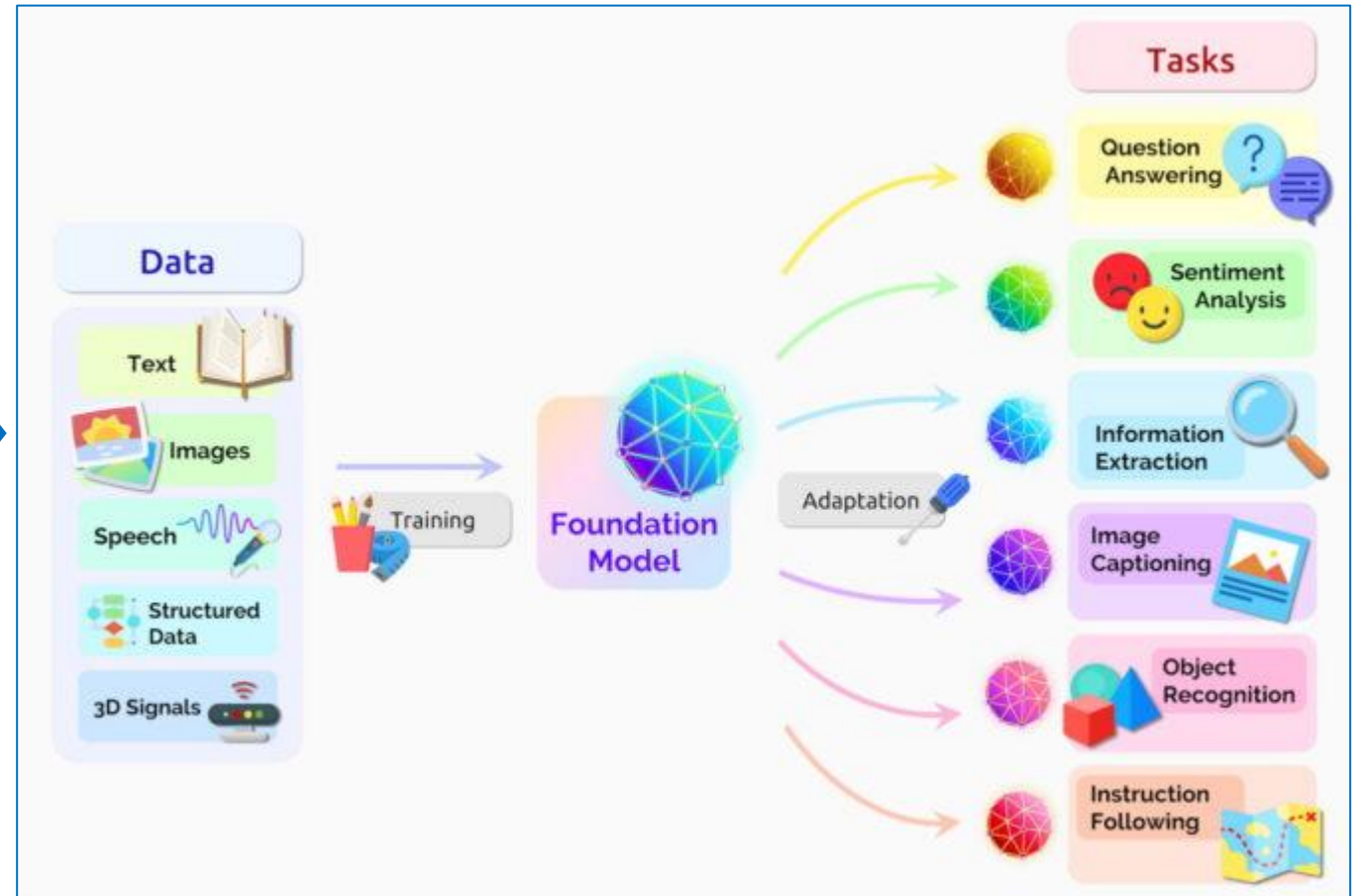
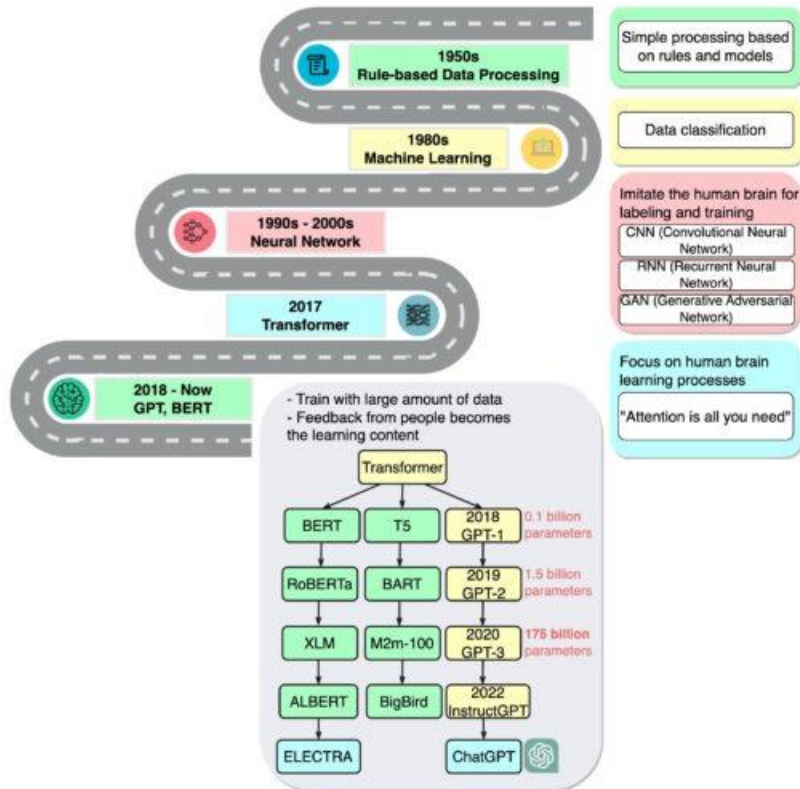
1. NLP와 LLM

- 간단하게 이해하는 LLM



1. NLP와 LLM

- **Foundation Model**
 - 비지도학습을 통해 학습된 AI신경망
 - 다양한 작업에 응용
 - Open source LLM



2. Transformer Review

- **Transformer 구조 리뷰**

- Encoder-only (BERT 계열): 양방향 context 이해 → classification, 임베딩 추출에 강점
- Decoder-only (GPT 계열): autoregressive 순차 생성에 특화
- Encoder-Decoder (T5, BART): 입력→출력 변환 task(번역, 요약)에 적합
- 핵심 구성요소: Multi-Head Self-Attention + Feed-Forward Network + Residual Connection + Layer Normalization
- Positional Encoding: 순서 정보 주입 방식 (sinusoidal / learned / RoPE)
- 제조 적용 예: 설비 로그 시퀀스 → encoder 표현학습, 불량보고서 생성 → decoder 활용

2. Transformer Review

- 토큰과 Attention

- Self-Attention 핵심 수식: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$
- Q/K/V를 "도서관 검색"으로 비유: Query(찾고 싶은 키워드) · Key(책 제목) · Value(책 내용)
- Multi-Head Attention: 여러 head가 문법적 관계·의미적 관계 등 서로 다른 관점을 병렬 학습
- Attention score = 토큰 간 연관도 → 가중합으로 새 representation 생성
- 시각화 시 주의: `attn_implementation="eager"` 설정 필요 (sdpa는 attention map 추출 불가)

2. Transformer Review

- 이미지와 시계열에서의 토큰화

- ViT: 이미지를 고정 크기 patch(예: 16×16)로 분할 → flatten → linear projection = 패치 임베딩 (텍스트 토큰과 동일한 역할)
- 시계열: 일정 구간(patch)을 하나의 토큰처럼 취급 — PatchTST, Moirai, Chronos 계열의 patch 기반 토큰화
- Chronos는 시계열 값을 quantize해서 "언어처럼" 이산 토큰화 → 기존 LLM 구조를 그대로 재사용
- 공통 원리: 모달리티가 달라도 "고정 단위로 쪼개서 임베딩 벡터화"하면 동일한 Transformer 구조에 입력 가능

2. Transformer Review

- **Decoder 생성 및 생성 파라미터(Temperature & Top_p)**

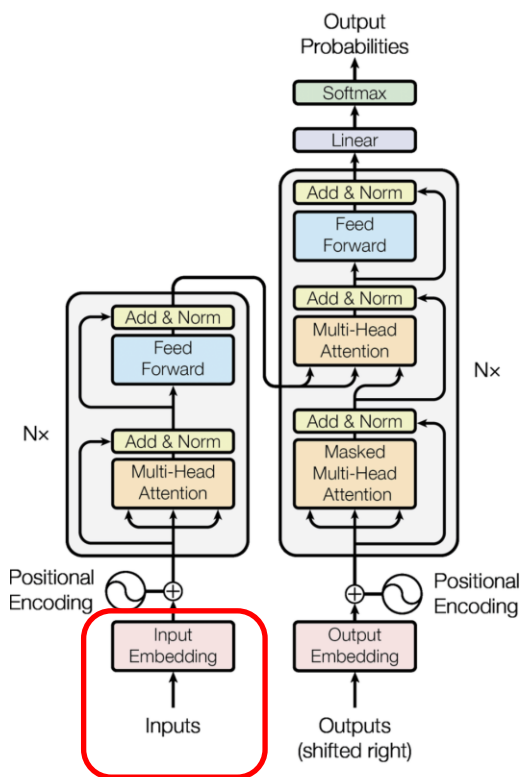
- Autoregressive 생성: 이전 토큰들을 조건으로 다음 토큰의 확률분포를 계산하고 반복 샘플링
- Greedy decoding(항상 최고확률 선택) vs Sampling(확률분포에서 추출)
- Temperature: 확률분포의 sharpness 조절 — 낮을수록 결정적/보수적, 높을수록 다양함/무작위
- Top-p(nucleus sampling): 누적확률 p 이내 후보만 남기고 샘플링 → 비현실적 토큰 배제
- Top-k와 차이: Top-k는 후보 개수 고정, Top-p는 확률분포에 따라 후보 개수가 가변적
- 제조 QA 챗봇 설계 가이드: 사실 기반 응답이 필요하므로 낮은 temperature(0.1~0.3) 권장

2. Transformer Review

- **주요 Transformer**

- Encoder-only: BERT, RoBERTa — 분류/임베딩 용도
- Decoder-only: GPT, Llama, Mistral, Gemma — 생성형 LLM
- Encoder-Decoder: T5, BART — 번역/요약 등 변환형 task
- Vision: ViT(균일 patch 분할) vs Swin(계층적 window attention, 고해상도 이미지에 유리)
- 시계열: Moirai(범용 사전학습 시계열 FM), Chronos(LLM 구조 재사용형), TimeFM(Google 시계열 foundation model)
- 제조 도메인 매핑: 결함 분류 → ViT/Swin, 설비 이상탐지 → 시계열 FM, 보고서 생성/QA → GPT 계열

2. Transformer Review



- Transformer Encoder에 대한 Input

- 토큰 8개: "나는 오늘 공장 라인 에서 불량 을 발견 "

Token	나는	오늘	공장	라인	에서	불량	을	발견
index	0	1	2	3	4	5	6	7

- Embedding: $d_{\text{model}}=4$

`nn.Embedding(vocab_size, d_model)`

- 초기값은 랜덤, 모형과 함께 학습, 문맥에 따라 변경(말(동물), 말(언어))
- Attention 을 거치며 업데이트됨 + 위치정보 반영

Token	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1	-0.35	0.62	0.45	0.79	0.04	0.09	-0.24
dim 1	0.64	-0.79	0.6	0.26	0.56	0.12	0.96	0.84
dim 2	-0.83	-0.03	0.56	0.48	0.07	0.71	0.27	0.46
dim 3	0.19	-0.35	0.52	0.02	-0.89	-0.78	0.33	-0.39

2. Transformer Review

• Positional Encoding

- Positional Encoding 반영 전

Token	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1	-0.35	0.62	0.45	0.79	0.04	0.09	-0.24
dim 1	0.64	-0.79	0.6	0.26	0.56	0.12	0.96	0.84
dim 2	-0.83	-0.03	0.56	0.48	0.07	0.71	0.27	0.46
dim 3	0.19	-0.35	0.52	0.02	-0.89	-0.78	0.33	-0.39

- Positional Encoding 값

- $PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$ ← 짝수 차원
 $PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$ ← 홀수 차원



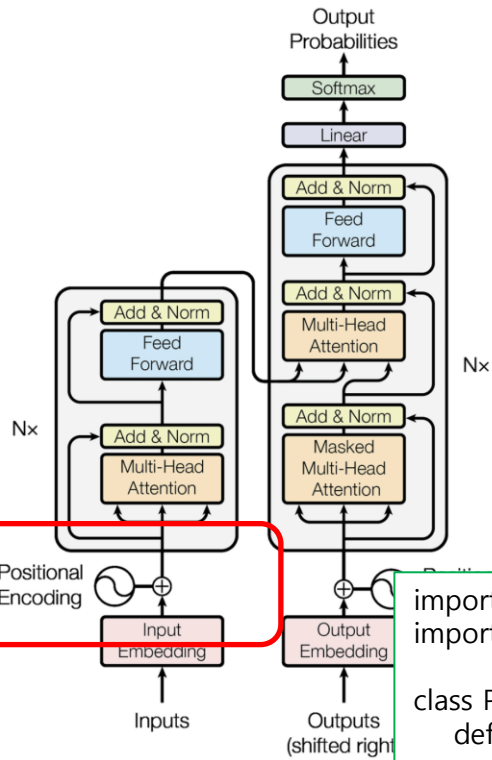
Token	공식	나는	오늘	공장	라인	에서	불량	을	발견
dim 0(짝수)	$\sin(pos/10000^{0/4})$	0	0.841	0.909	0.141	-0.757	-0.959	-0.279	0.657
dim 1(홀수)	$\cos(pos/10000^{0/4})$	1	0.54	-0.416	-0.99	-0.654	0.284	0.96	0.754
dim 2(짝수)	$\sin(pos/10000^{2/4})$	0	0.01	0.02	0.03	0.04	0.05	0.06	0.07
dim 3(홀수)	$\cos(pos/10000^{2/4})$	1	1	1	1	0.999	0.999	0.998	0.998

2. Transformer Review

- **Positional Encoding:** 값이 -1~1로 제한, 학습 때 없던 긴 시퀀스에도 일반화 가능

- Positional Encoding 반영 후

Token	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1	0.492	1.527	0.593	0.032	-0.922	-0.189	0.419
dim 1	1.639	-0.251	0.184	-0.731	-0.097	0.409	1.924	1.59
dim 2	-0.828	-0.024	0.58	0.507	0.109	0.763	0.328	0.533
dim 3	1.195	0.645	1.516	1.018	0.111	0.214	1.326	0.605



```
import torch
import math

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=512):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(max_len).unsqueeze(1) # (max_len, 1)
        div = torch.exp(torch.arange(0, d_model, 2) *
                        (-math.log(10000.0) / d_model)) # 분모 계산

        pe[:, 0::2] = torch.sin(pos * div) # 짝수 차원 sin
        pe[:, 1::2] = torch.cos(pos * div) # 홀수 차원 cos

        self.register_buffer('pe', pe) # 학습 안 되는 상수로 등록

    def forward(self, x):
        return x + self.pe[:x.size(1)] # 임베딩에 그냥 더하기
```

- "불량"토큰의 예: 임베딩 + PE

	dim 0	dim 1	dim 2	dim 3
Emb	0.037	0.125	0.713	-0.785
PE	-0.959	0.284	0.05	0.999
Result	-0.922	0.409	0.763	0.214

별도 함수 없이 직접 구현

2. Transformer Review

- **Q, K, V 생성:** 입력에 각 W_q, W_k, W_v 행렬을 곱해 Q, K, V 생성 (초기 랜덤, 학습되는 파라미터)

- 각 행렬: $d_{\text{model}} \times d_{\text{model}}$

- Query

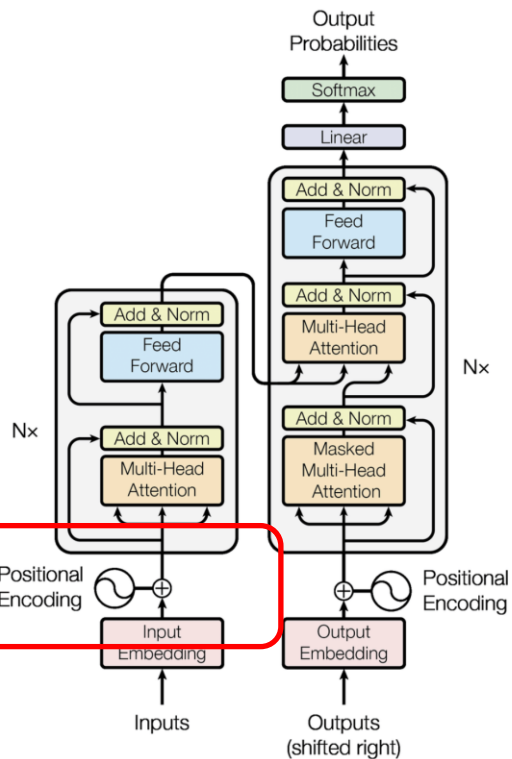
차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.99	0.41	-0.39	0.19	0.3	1.3	-0.34	0.17
dim 1	-0.44	0.07	1.25	0.67	1.25	0.06	0.37	1.38
dim 2	0.29	-0.28	-0.64	-0.35	-1.01	0.04	0.03	-0.86
dim 3	-0.55	-0.03	1.16	0.58	0.4	-0.06	0.37	0.79

- Key

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	1.15	-0.83	-0.95	-0.19	1.06	1.4	0.84	0.73
dim 1	0.58	0.31	-1.38	-0.8	-0.73	-0.42	-0.79	-1.28
dim 2	-0.55	0.52	-1.11	-0.15	0.53	1.36	-0.62	-0.04
dim 3	1.85	-0.76	-1.65	-0.63	1.2	1.08	0.55	0.41

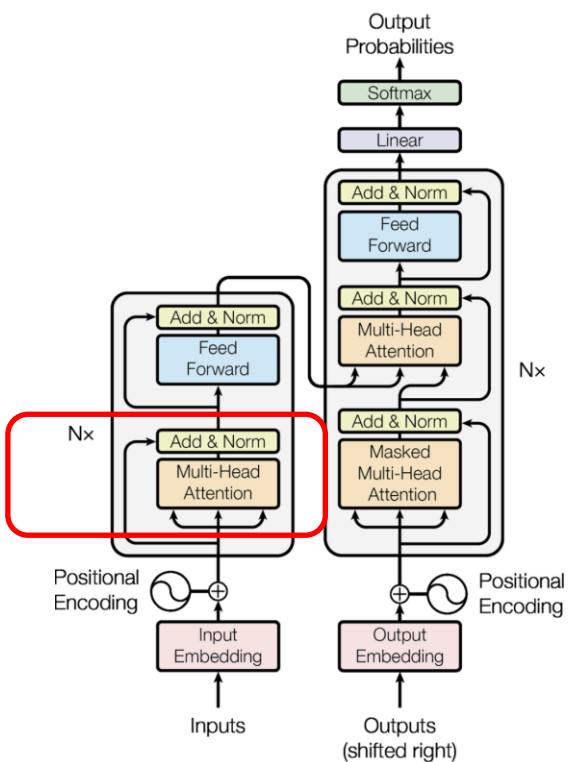
- Value

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.1	-0.35	0.2	0.13	-0.5	0.21	0.4	-0.09
dim 1	1.24	-0.35	-1.58	-0.94	-1.13	-0.31	-0.2	-1.43
dim 2	-0.47	-0.1	0.48	0.31	-0.12	0.28	0.25	0.23
dim 3	1.04	-0.79	-0.54	-0.22	0.15	0.5	0.73	0.15



행렬: `nn.Linear(d_model, d_model)`

2. Transformer Review

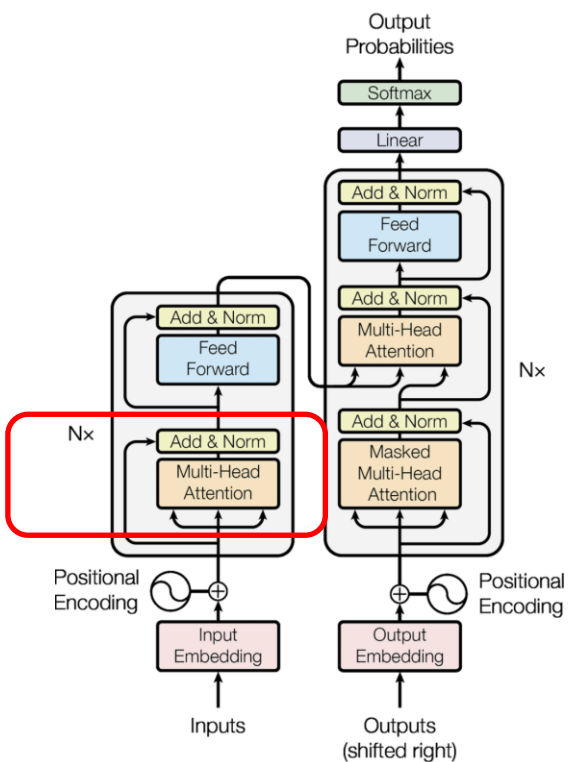


- **Attention Score:** $QK^T / \sqrt{d_{\text{model}}}$, 각 토큰이 다른 토큰에 얼마나 주목하는지의 점수
 - 결과: 토큰 수 X 토큰 수

쿼리/키	나는	오늘	공장	라인	에서	불량	을	발견
나는	-1.29	0.63	1.07	0.42	-0.62	-0.7	-0.48	-0.2
오늘	0.31	-0.22	-0.06	-0.04	0.1	0.07	0.22	0.1
공장	1.39	-0.25	-1.28	-0.78	-0.14	-0.34	-0.14	-0.69
라인	0.94	-0.29	-0.84	-0.44	0.11	0.07	0.08	-0.23
에서	1.18	-0.35	-0.77	-0.58	-0.32	-0.52	0.06	-0.59
불량	0.7	-0.5	-0.63	-0.13	0.64	0.89	0.49	0.42
을	0.25	0.07	-0.42	-0.23	-0.09	-0.1	-0.2	-0.29
발견	1.47	-0.38	-1.21	-0.75	-0.17	-0.33	0.01	-0.64

행렬: `nn.Linear(d_model, d_model)`

2. Transformer Review

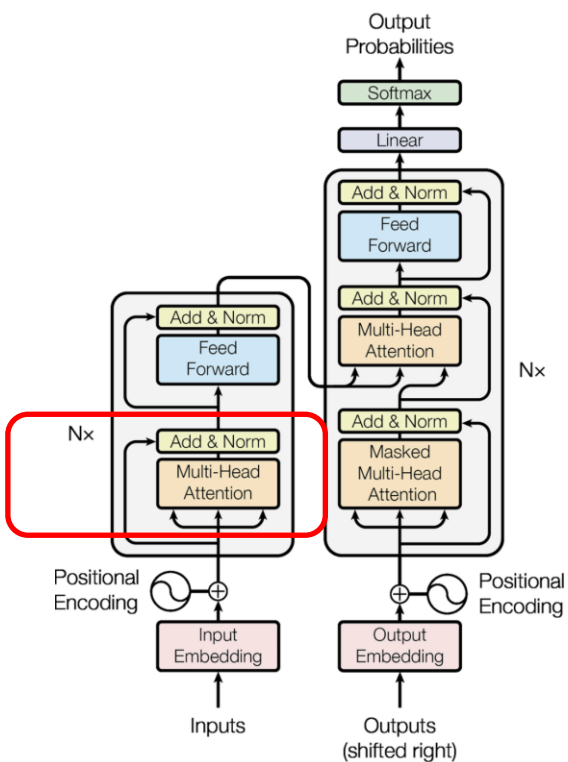


- **Attention Weight:** softmax 적용된 score, 행별 합이 1인 가중치, 높을 수록 토큰에 더 집중

쿼리/키	나는	오늘	공장	라인	에서	불량	을	발견
나는	0.03	0.21	0.32	0.17	0.06	0.05	0.07	0.09
오늘	0.16	0.09	0.11	0.11	0.13	0.12	0.14	0.13
공장	0.47	0.09	0.03	0.05	0.1	0.08	0.1	0.06
라인	0.3	0.09	0.05	0.08	0.13	0.13	0.13	0.09
에서	0.41	0.09	0.06	0.07	0.09	0.08	0.13	0.07
불량	0.17	0.05	0.05	0.08	0.16	0.21	0.14	0.13
을	0.18	0.15	0.09	0.11	0.13	0.13	0.11	0.1
발견	0.49	0.08	0.03	0.05	0.09	0.08	0.11	0.06

행렬: `nn.Linear(d_model, d_model)`

2. Transformer Review



• Attention 출력 및 MHA

- Attention Weight $X V$: 문맥을 반영한 각 토큰의 새로운 표현
- 원래의 PE 적용 토큰 임베딩

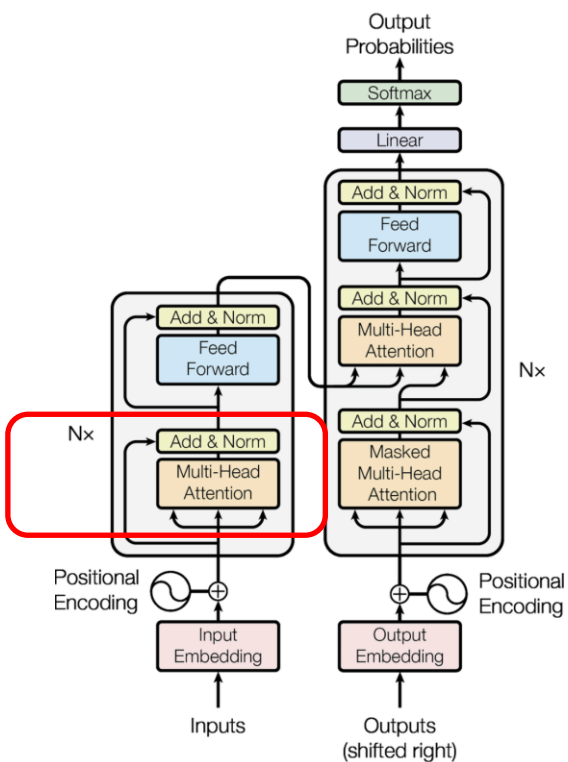
Token	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1	0.49	1.53	0.59	0.03	-0.92	-0.19	0.42
dim 1	0.64	-0.78	0.62	0.29	0.6	0.17	1.02	0.91
dim 2	-0.83	-0.03	0.56	0.48	0.07	0.71	0.27	0.46
dim 3	0.19	-0.35	0.52	0.02	-0.89	-0.78	0.33	-0.39

- Attention Output: Attention 적용 토큰 임베딩

Token	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	0.01	-0.01	-0.06	-0.03	-0.03	-0.01	-0.04	-0.05
dim 1	-0.93	-0.51	0.21	-0.16	0.06	-0.42	-0.43	0.25
dim 2	0.22	0.09	-0.15	-0.03	-0.09	0.07	0.05	-0.16
dim 3	-0.25	0.21	0.53	0.39	0.47	0.34	0.18	0.56

2. Transformer Review

행렬: `nn.MultiheadAttention(d_model=4, num_heads=2)`



- MHA 헤드 분리: $d_{\text{model}}=4$ 를 2개의 헤드(각 $\text{dim}=2$)로 나눠 Q, K, V 계산

- 헤드1의 Q, K, V

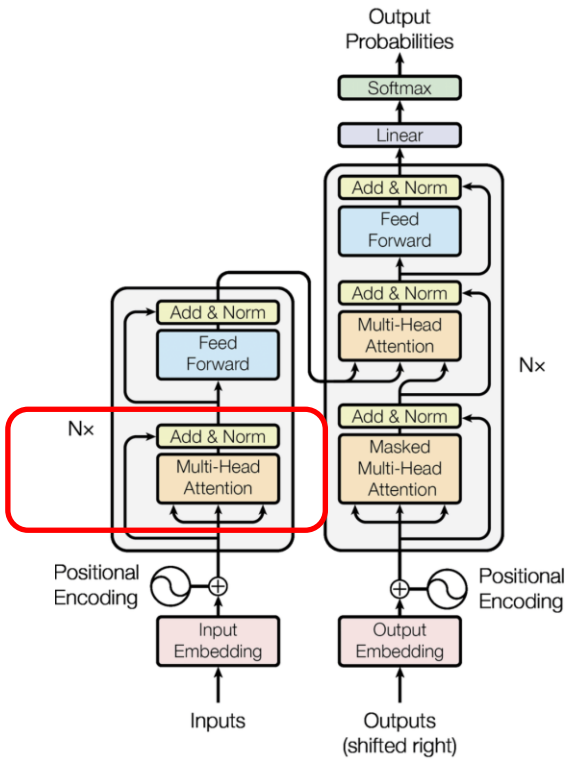
Q	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-2.04	-0.402	-0.753	0.106	0.081	0.667	-1.285	-0.65
dim 1	-0.497	-0.356	0.054	-0.977	-0.147	-0.595	0.237	1.06
K	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	1.38	-1.008	-1.943	-1.695	-0.17	0.994	0.996	0.703
dim 1	0.082	0.151	-0.834	0.184	0.009	-0.392	-1.264	-1.599
V	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	0.483	0.228	0.796	0.736	0.121	0.831	1.005	0.532
dim 1	1.624	0.238	-0.525	0.397	0.051	0.397	0.187	-0.948

- 헤드2의 Q, K, V

Q	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1.133	-0.78	-3.064	-1.146	-0.117	-0.238	-2.747	-2.396
dim 1	0.62	0.578	0.559	1.149	0.145	0.243	0.006	-0.779
K	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.903	0.183	-0.561	0.015	-0.04	-1.192	-1.92	-1.578
dim 1	-0.349	0.117	0.409	0.177	0.015	-0.192	-0.062	0.124
V	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	0.246	0.077	-0.402	0.15	0.018	-0.006	-0.425	-0.728
dim 1	-2.623	0.119	-0.24	0.297	0.021	-1.126	-2.607	-1.545

2. Transformer Review

행렬: $\text{attn_out}, \text{attn_w} = \text{nn.MultiheadAttention}(\dots)(Q, K, V)$



• MHA Score, Weight (1/2)

• 헤드1의 Score

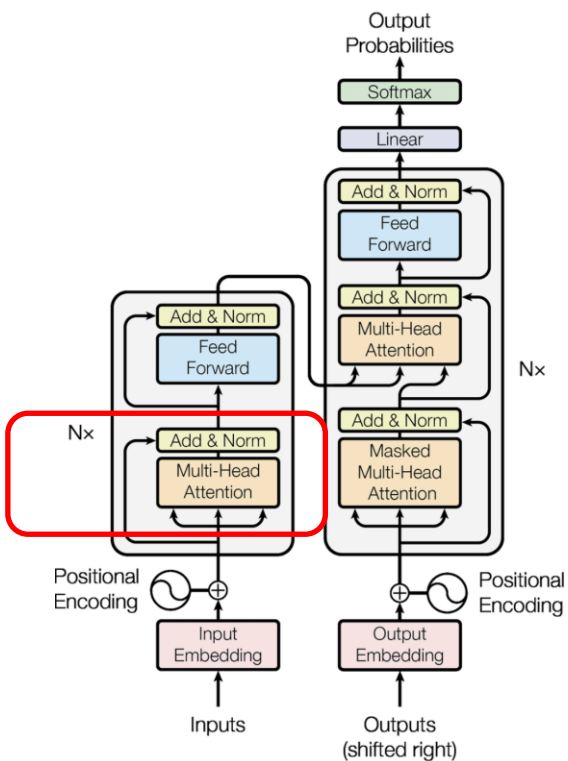
Q/K	나는	오늘	공장	라인	에서	불량	을	발견
나는	-2.019	1.401	3.096	2.38	0.242	-1.296	-0.993	-0.452
오늘	-0.413	0.249	0.762	0.435	0.046	-0.184	0.035	0.203
공장	-0.732	0.542	1.003	0.91	0.091	-0.544	-0.579	-0.435
라인	0.047	-0.18	0.431	-0.254	-0.019	0.345	0.948	1.157
에서	0.071	-0.073	-0.025	-0.116	-0.011	0.098	0.188	0.206
불량	0.616	-0.539	-0.566	-0.877	-0.084	0.634	1.002	1.004
을	-1.24	0.941	1.626	1.571	0.156	-0.969	-1.117	-0.907
발견	-0.573	0.576	0.268	0.917	0.085	-0.751	-1.405	-1.522

• 헤드1 Weight

Q/K	나는	오늘	공장	라인	에서	불량	을	발견
나는	0.003	0.102	0.557	0.272	0.032	0.007	0.009	0.016
오늘	0.068	0.131	0.219	0.158	0.107	0.085	0.106	0.125
공장	0.047	0.167	0.265	0.241	0.106	0.056	0.054	0.063
라인	0.085	0.068	0.125	0.063	0.079	0.114	0.209	0.257
에서	0.128	0.111	0.116	0.106	0.118	0.131	0.144	0.146
불량	0.159	0.05	0.049	0.036	0.079	0.161	0.233	0.234
을	0.019	0.171	0.338	0.32	0.078	0.025	0.022	0.027
발견	0.069	0.218	0.16	0.306	0.133	0.058	0.03	0.027

2. Transformer Review

행렬: `attn_out, attn_w = nn.MultiheadAttention(...)(Q, K, V)`



• MHA Score, Weight (2/2)

• 헤드2의 Score

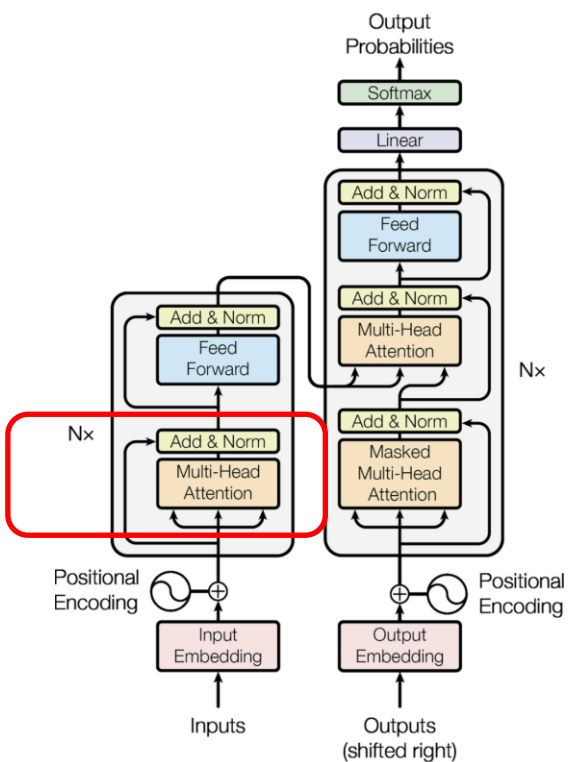
Q/K	나는	오늘	공장	라인	에서	불량	을	발견
나는	0.57	-0.095	0.629	0.066	0.039	0.871	1.511	1.319
오늘	0.355	-0.053	0.477	0.064	0.028	0.579	1.034	0.921
공장	1.818	-0.35	1.377	0.037	0.093	2.507	4.135	3.468
라인	0.448	-0.053	0.787	0.132	0.045	0.81	1.505	1.379
에서	0.039	-0.003	0.088	0.017	0.005	0.079	0.152	0.143
불량	0.092	-0.011	0.165	0.028	0.009	0.168	0.312	0.287
을	1.753	-0.355	1.091	-0.028	0.078	2.315	3.729	3.066
발견	1.722	-0.374	0.725	-0.123	0.06	2.125	3.287	2.605

• 헤드2 Weight

Q/K	나는	오늘	공장	라인	에서	불량	을	발견
나는	0.102	0.052	0.108	0.062	0.06	0.138	0.262	0.216
오늘	0.108	0.072	0.122	0.081	0.078	0.135	0.213	0.19
공장	0.051	0.006	0.033	0.009	0.009	0.102	0.522	0.268
라인	0.089	0.054	0.125	0.065	0.059	0.128	0.256	0.225
에서	0.122	0.117	0.128	0.119	0.118	0.127	0.136	0.135
불량	0.119	0.108	0.128	0.112	0.11	0.129	0.149	0.145
을	0.068	0.008	0.035	0.011	0.013	0.12	0.491	0.253
발견	0.095	0.012	0.035	0.015	0.018	0.142	0.454	0.229

2. Transformer Review: Encoder

Wo = out_proj, nn.MultiheadAttention 내부 자동 처리



- MHA Concat + Wo를 통한 MHA 출력: 헤드1+헤드2의 concat, Wo곱하는 계산

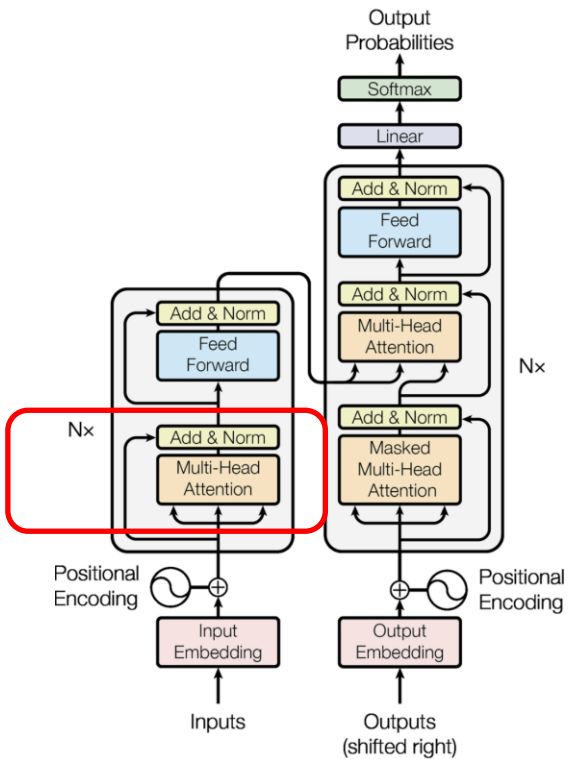
- 헤드1+헤드2 concat (dim=4)

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	0.696	0.61	0.596	0.653	0.603	0.656	0.619	0.544
dim 1	-0.164	0.03	0.051	-0.042	0.162	0.148	0.014	0.211
dim 2	-0.273	-0.233	-0.416	-0.287	-0.149	-0.165	-0.388	-0.348
dim 3	-1.44	-1.279	-2.028	-1.396	-1.005	-1.052	-1.988	-1.949

- $\times W_o \rightarrow$ MHA 출력

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.255	-0.324	-0.875	-0.327	-0.206	-0.18	-0.807	-0.954
dim 1	-0.299	-0.357	-0.329	-0.331	-0.44	-0.461	-0.331	-0.402
dim 2	-1.162	-1.187	-1.55	-1.189	-1.181	-1.229	-1.535	-1.657
dim 3	-0.784	-0.721	-1.045	-0.766	-0.624	-0.654	-1.032	-1.032

2. Transformer Review: Encoder



- Residual Connection + Layer Normalization (1/2)

- 원래의 입력 (PE 적용 임베딩, X)

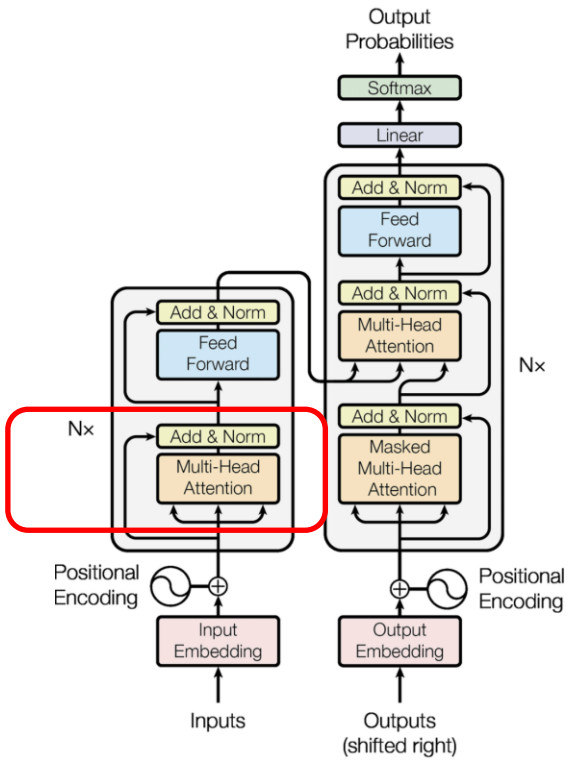
차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1	0.492	1.527	0.593	0.032	-0.922	-0.189	0.419
dim 1	1.639	-0.251	0.184	-0.731	-0.097	0.409	1.924	1.59
dim 2	-0.828	-0.024	0.58	0.507	0.109	0.763	0.328	0.533
dim 3	1.195	0.645	1.516	1.018	0.111	0.214	1.326	0.605

- MHA 출력

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.255	-0.324	-0.875	-0.327	-0.206	-0.18	-0.807	-0.954
dim 1	-0.299	-0.357	-0.329	-0.331	-0.44	-0.461	-0.331	-0.402
dim 2	-1.162	-1.187	-1.55	-1.189	-1.181	-1.229	-1.535	-1.657
dim 3	-0.784	-0.721	-1.045	-0.766	-0.624	-0.654	-1.032	-1.032

```
self.norm1 = nn.LayerNorm(d_model)
out = self.norm1(x + attn_output)
```

2. Transformer Review: Encoder



• Residual Connection + Layer Normalization (2/2)

- Residual: $X + \text{MHA}$

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1.255	0.168	0.652	0.266	-0.174	-1.102	-0.996	-0.535
dim 1	1.34	-0.608	-0.145	-1.062	-0.537	-0.052	1.593	1.188
dim 2	-1.99	-1.211	-0.97	-0.682	-1.072	-0.466	-1.207	-1.124
dim 3	0.411	-0.076	0.471	0.252	-0.513	-0.44	0.294	-0.427

- Layer Normalization

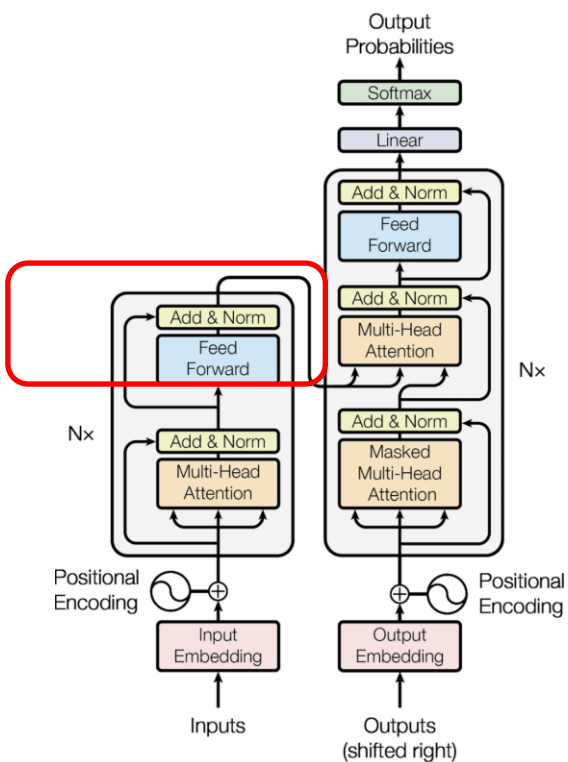
차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.67	1.132	1.025	0.986	1.246	-1.561	-0.817	-0.361
dim 1	1.301	-0.332	-0.232	-1.299	0.115	1.231	1.489	1.647
dim 2	-1.228	-1.47	-1.533	-0.645	-1.551	0.13	-1.004	-1.048
dim 3	0.595	0.672	0.74	0.962	0.19	0.199	0.332	-0.235

```
self.norm1 = nn.LayerNorm(d_model)
out = self.norm1(x + attn_output)
```

$$\gamma \frac{V - \mu}{\sigma} + \beta$$

scale bias

2. Transformer Review: Encoder



- **FFN: Linear(4→8) → ReLU → Linear(8→4) → Residual + LayerNorm**

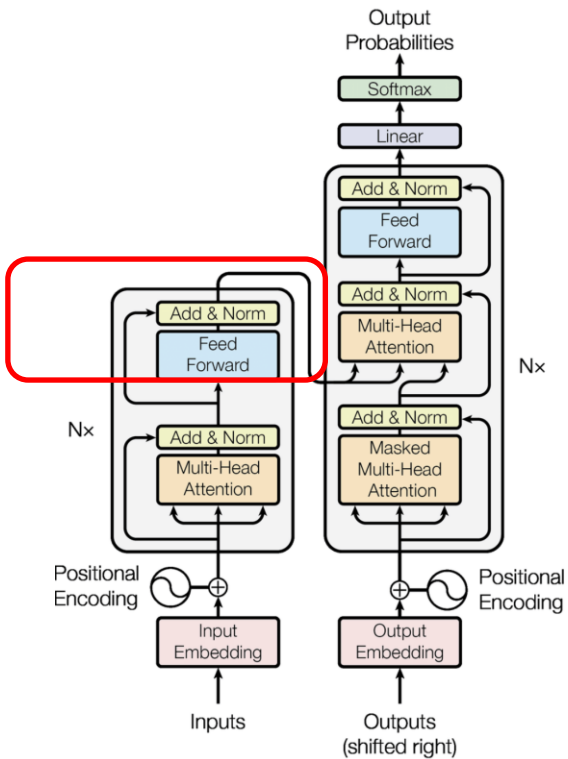
- FFN 중간층 (dim=8, ReLU 후)

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	2.681	0.692	0.933	0	0.511	1.908	2.497	1.802
dim 1	0.289	0.891	0.905	0.915	0.659	0	0.07	0
dim 2	0	0	0	0	0	0.788	0	0
dim 3	0	1.043	0.984	1.23	0.851	0	0	0
dim 4	0.71	0.272	0.324	0	0.347	0.38	0.686	0.653
dim 5	0.284	0	0	0	0.272	0.157	0.397	0.698
dim 6	0	0	0	0.162	0	0	0	0
dim 7	0	0	0	0	0	0	0	0

- +Residual

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1.015	1.366	1.232	1.226	1.202	-2.216	-1.233	-0.998
dim 1	0.664	0.623	0.648	-0.139	0.675	0.465	0.651	0.756
dim 2	1.4	-1.121	-0.982	-1.158	-0.899	1.781	1.701	1.457
dim 3	-1.065	0.487	0.368	1.371	0.067	-1.362	-1.067	-1.283

2. Transformer Review: Encoder

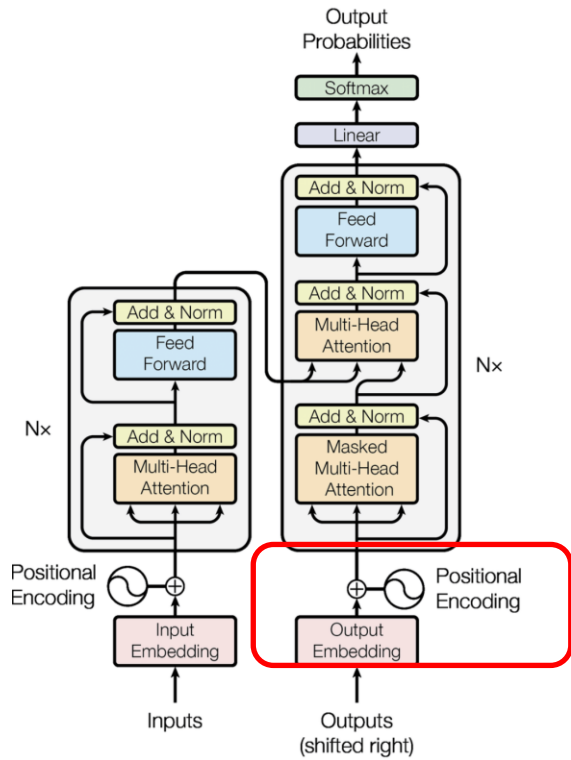


- FFN: $\text{Linear}(4 \rightarrow 8) \rightarrow \text{ReLU} \rightarrow \text{Linear}(8 \rightarrow 4) \rightarrow \text{Residual} + \text{LayerNorm}$
- LayerNorm 후 $\rightarrow$ Encoder 최종 출력

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.947	1.132	1.127	0.867	1.205	-1.209	-1.02	-0.849
dim 1	0.625	0.313	0.408	-0.447	0.53	0.512	0.522	0.669
dim 2	1.315	-1.61	-1.6	-1.427	-1.485	1.357	1.381	1.276
dim 3	-0.993	0.163	0.063	1.007	-0.248	-0.66	-0.884	-1.096

```
self.ff = nn.Sequential(nn.Linear(d_model,d_ff), nn.ReLU(), nn.Linear(d_ff,d_model))
self.norm2 = nn.LayerNorm(d_model)
out = self.norm2(x + self.ff(x))
```

2. Transformer Review: Decoder



• Decoder 입력 + Masked MHA

• Decoder 임베딩(출력토큰)

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.983	-0.268	0.432	0.286	0.5	-0.318	0.891	-0.777
dim 1	0.544	-0.267	0.718	-0.421	-0.826	-0.216	-0.174	0.432
dim 2	0.322	0.642	0.701	-0.651	-0.37	0.093	0.695	-0.362
dim 3	0.712	0.354	0.648	-0.636	0.201	-0.273	-0.778	0.371

• +PE -> Decoder 입력

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.983	0.573	1.341	0.427	-0.257	-1.277	0.612	-0.12
dim 1	1.544	0.273	0.302	-1.411	-1.48	0.068	0.786	1.186
dim 2	0.322	0.652	0.721	-0.621	-0.33	0.143	0.755	-0.292
dim 3	1.712	1.354	1.648	0.364	1.2	0.726	0.22	1.369

causal_mask 생성

```
mask = torch.triu(torch.ones(N,N), diagonal=1).bool()
nn.MultiheadAttention(...)(Q,K,V, attn_mask=mask)
```


2. Transformer Review: Decoder

Causal Mask 예:

[[F, T, T, T],
[F, F, T, T],
[F, F, F, T],
[F, F, F, F]]

Masking의 이유:

과거/현재의 토큰을 보고 미래 토큰 예측을 해야 인과관계가 유지
미래를 보고 현재를 예측하는 것은 인과관계 위반임

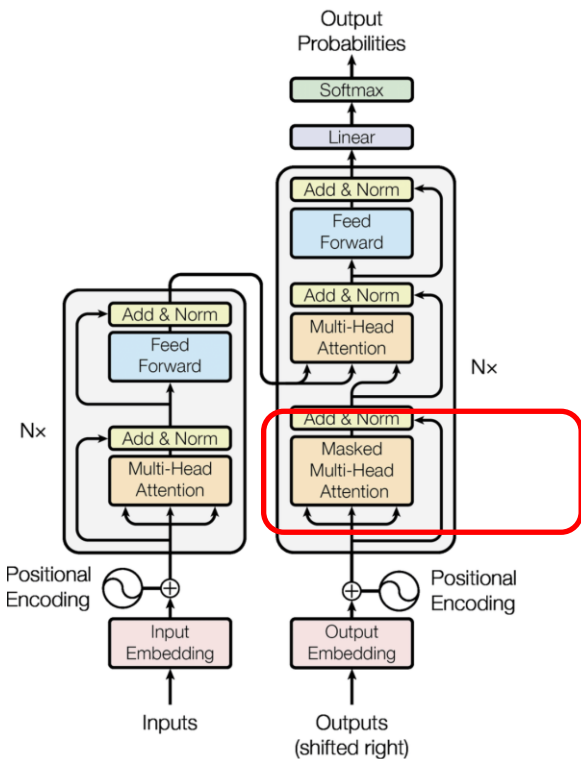
Masked MHA

- Masked Score: $-\infty$, 미래 토큰 차단 / 미래 토큰은 이전 토큰의 인과

Q/K	나는	오늘	공장	라인	에서	불량	을	발견
나는	-0.322	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
오늘	-0.528	-1.849	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
공장	-0.772	-2.547	-4.095	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
라인	0.336	0.713	1.176	0.952	$-\infty$	$-\infty$	$-\infty$	$-\infty$
에서	0.348	0.378	0.665	0.227	0.164	$-\infty$	$-\infty$	$-\infty$
불량	0.19	0.14	0.261	-0.014	-0.081	-0.114	$-\infty$	$-\infty$
을	-0.565	-1.501	-2.44	-2.233	-2.569	0.185	0.438	$-\infty$
발견	-0.238	-1.065	-1.691	-1.85	-2.205	0.016	0.477	-0.98

- Masked Weight(softmax 후 미래=0)

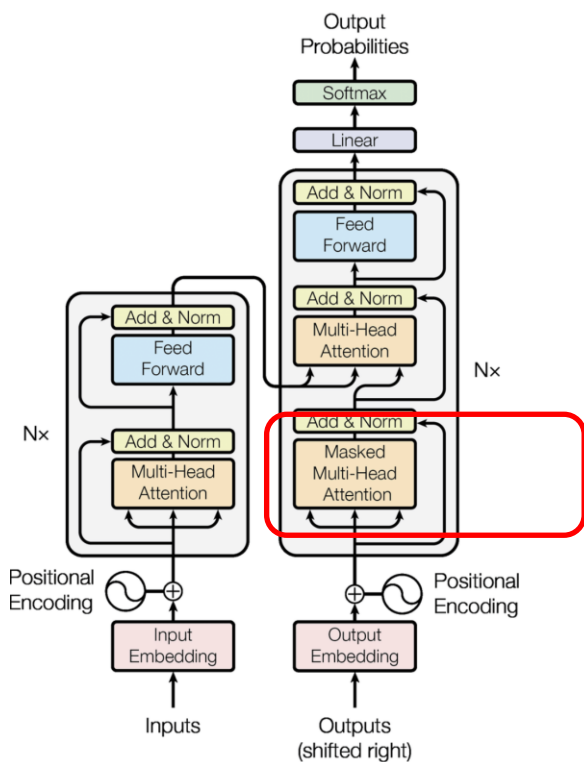
Q/K	나는	오늘	공장	라인	에서	불량	을	발견
나는	1	0.000	0.000	0.000	0.000	0.000	0.000	0.000
오늘	0.789	0.211	0.000	0.000	0.000	0.000	0.000	0.000
공장	0.83	0.141	0.03	0.000	0.000	0.000	0.000	0.000
라인	0.151	0.22	0.35	0.279	0.000	0.000	0.000	0.000
에서	0.195	0.201	0.268	0.173	0.162	0.000	0.000	0.000
불량	0.187	0.178	0.201	0.153	0.143	0.138	0.000	0.000
을	0.149	0.058	0.023	0.028	0.02	0.315	0.406	0.000
발견	0.172	0.075	0.04	0.034	0.024	0.222	0.351	0.082



"나는" 자기자신만 보고
"발견"은 모든 토큰을 볼 수 있음

`nn.MultiheadAttention(d_model=4,
num_heads=2)`

2. Transformer Review: Decoder



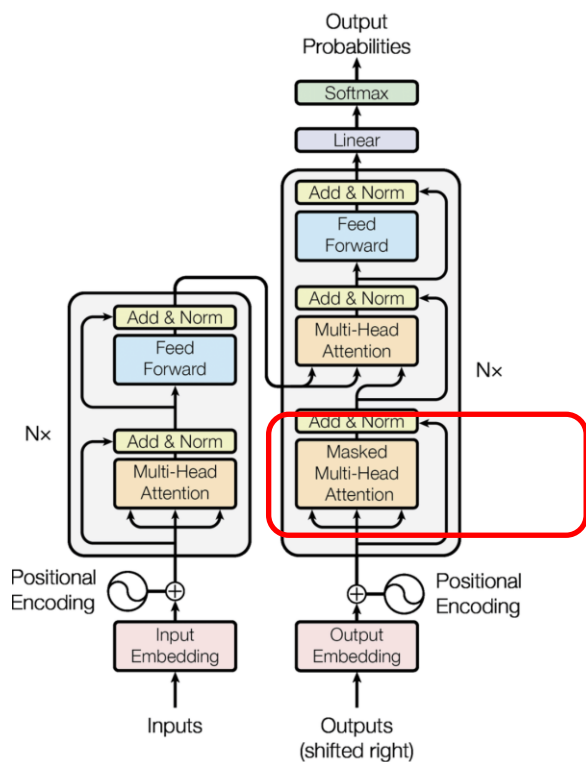
• Masked MHA Concat

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-3.31	-2.704	-2.81	-0.36	-0.608	-0.806	-0.95	-1.084
dim 1	-0.377	-0.459	-0.461	-0.366	-0.112	0.066	-0.359	-0.43
dim 2	0.313	0.705	1.183	1.193	1.14	0.627	0.644	0.764
dim 3	2.12	1.7	1.44	1.136	1.194	0.968	0.858	1.094

• $\times W_o \rightarrow$ Masked MHA 출력

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-3.83	-3.167	-3.134	-1.058	-1.23	-1.191	-1.321	-1.577
dim 1	2.297	1.909	2.065	0.465	0.742	0.811	0.698	0.806
dim 2	1.211	0.724	0.583	-0.535	-0.259	0.146	-0.019	-0.047
dim 3	-0.698	-0.439	-0.346	0.019	-0.162	-0.287	-0.067	-0.088

2. Transformer Review: Decoder



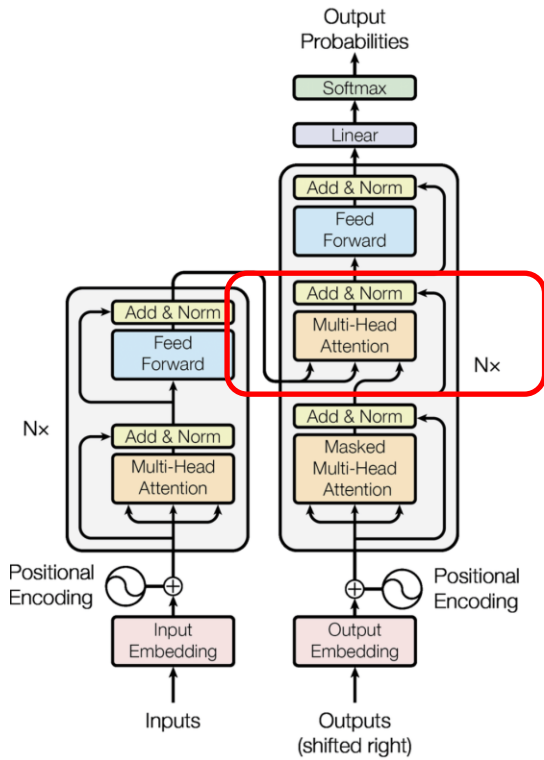
- **+Residual**

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-4.813	-2.594	-1.793	-0.631	-1.487	-2.468	-0.709	-1.697
dim 1	3.841	2.182	2.367	-0.946	-0.738	0.879	1.484	1.992
dim 2	1.533	1.376	1.304	-1.156	-0.589	0.289	0.736	-0.339
dim 3	1.014	0.915	1.302	0.383	1.038	0.439	0.153	1.281

- **Layer Normalization**

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1.633	-1.678	-1.663	-0.074	-1.133	-1.708	-1.401	-1.399
dim 1	1.081	0.938	1.01	-0.607	-0.319	0.83	1.33	1.174
dim 2	0.357	0.496	0.327	-0.962	-0.157	0.382	0.399	-0.452
dim 3	0.195	0.244	0.326	1.643	1.609	0.496	-0.328	0.678

2. Transformer Review: Decoder



- **Decoder Cross-Attention:** Decoder Q × Encoder K,V → 문맥 참조. Residual + LayerNorm

- Cross-Attention Score

(Q: 디코더,
K: 인코더)

Q/K	나는	오늘	공장	라인	에서	불량	을	발견
나는	0.233	0.649	0.734	-0.194	0.959	-0.091	0.101	0.345
오늘	0.36	0.708	0.812	-0.296	1.073	-0.004	0.206	0.485
공장	0.301	0.741	0.841	-0.245	1.1	-0.067	0.148	0.428
라인	0.469	1.005	1.144	-0.329	1.45	0.044	0.264	0.618
에서	0.909	1.374	1.596	-0.697	2.086	0.254	0.596	1.136
불량	0.454	0.871	0.999	-0.365	1.314	0.018	0.264	0.604
을	-0.081	0.243	0.257	0.048	0.333	-0.211	-0.116	-0.038
발견	0.027	0.819	0.895	-0.007	1.118	-0.323	-0.108	0.149

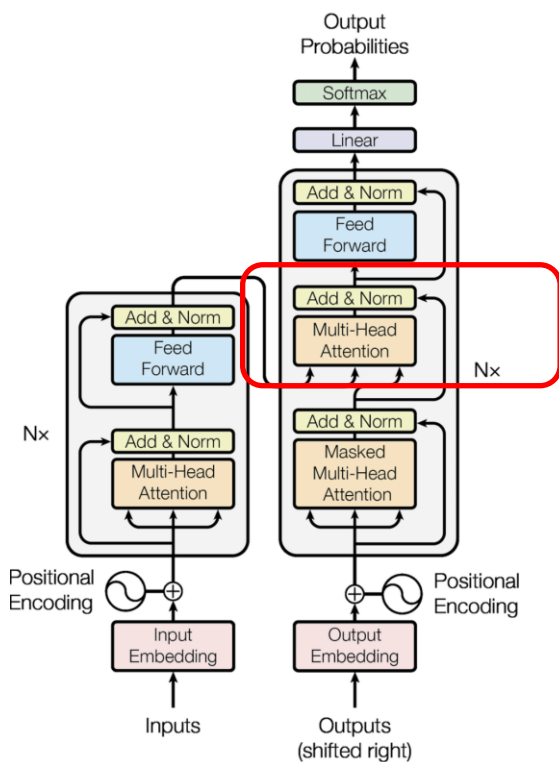
- Cross-Attention

Weight

Q/K	나는	오늘	공장	라인	에서	불량	을	발견
나는	0.104	0.158	0.172	0.068	0.215	0.075	0.091	0.116
오늘	0.108	0.153	0.17	0.056	0.221	0.075	0.093	0.123
공장	0.102	0.159	0.176	0.059	0.228	0.071	0.088	0.116
라인	0.096	0.164	0.188	0.043	0.256	0.063	0.078	0.111
에서	0.095	0.151	0.189	0.019	0.308	0.049	0.069	0.119
불량	0.103	0.157	0.178	0.046	0.244	0.067	0.085	0.12
을	0.107	0.148	0.15	0.122	0.162	0.094	0.104	0.112
발견	0.082	0.18	0.195	0.079	0.243	0.058	0.071	0.092

```
nn.TransformerDecoderLayer(d_model, nhead)
# query=decoder, key=encoder_out, value=encoder_out
# FFN + Residual + Norm 자동 포함
```

2. Transformer Review: Decoder



- Cross-attention 출력

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	0.177	0.136	0.194	0.257	0.274	0.19	0.131	0.396
dim 1	0.389	0.428	0.388	0.365	0.384	0.405	0.363	0.239
dim 2	-0.858	-0.846	-0.896	-0.997	-1.09	-0.927	-0.69	-1.054
dim 3	-0.536	-0.505	-0.571	-0.673	-0.748	-0.59	-0.399	-0.79

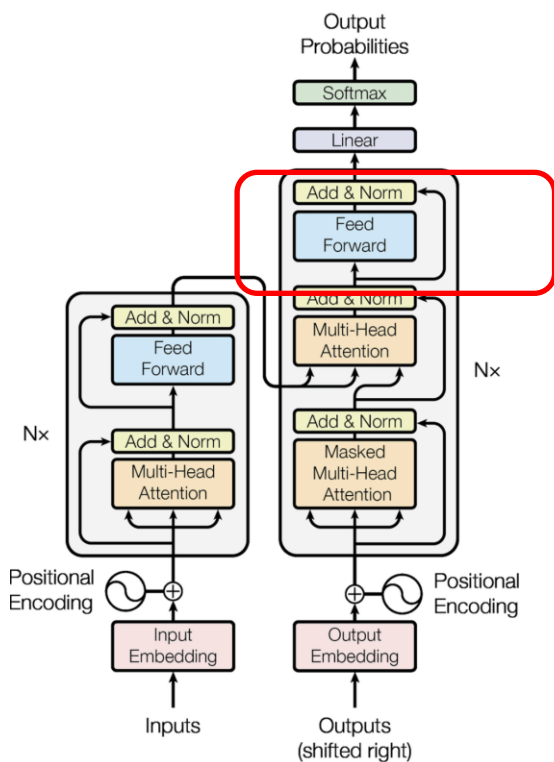
- + Residual

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1.456	-1.542	-1.469	0.183	-0.859	-1.518	-1.27	-1.003
dim 1	1.47	1.366	1.398	-0.242	0.065	1.235	1.693	1.413
dim 2	-0.501	-0.35	-0.569	-1.959	-1.247	-0.545	-0.291	-1.506
dim 3	-0.341	-0.261	-0.245	0.97	0.861	-0.094	-0.727	-0.112

- Layer Normalization

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1.181	-1.301	-1.203	0.415	-0.688	-1.3	-1.002	-0.632
dim 1	1.585	1.511	1.562	0.019	0.439	1.48	1.647	1.547
dim 2	-0.278	-0.148	-0.335	-1.583	-1.161	-0.318	-0.127	-1.086
dim 3	-0.127	-0.062	-0.023	1.149	1.41	0.138	-0.517	0.171

2. Transformer Review: Decoder



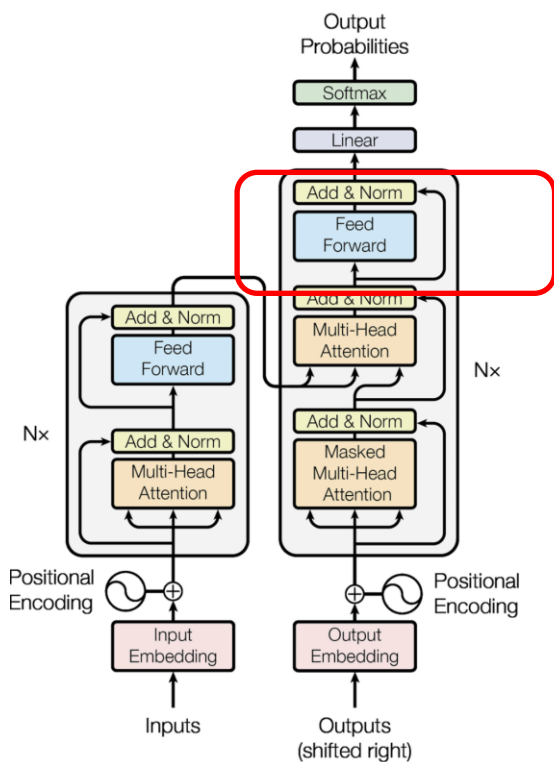
- **Decoder FFN + 최종 출력:** FFN($4 \rightarrow 8 \rightarrow 4$, ReLU) $\rightarrow$ Residual + LayerNorm $\rightarrow$ 디코더 최종 출력
- Decoder FFN 중간(dim=8, ReLU)

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	0	0	0	0	0	0	0.137	0.188
dim 1	1.439	1.552	1.336	0	0	1.28	1.7	0.215
dim 2	0.532	0.572	0.556	0.09	0.526	0.605	0.414	0.417
dim 3	1.286	1.379	1.321	0	0.956	1.414	1.074	0.902
dim 4	0	0	0.044	1.236	1.265	0.146	0	0.402
dim 5	0	0	0	0.774	1.281	0	0	0
dim 6	0	0	0	0.065	0	0	0	0
dim 7	1.691	1.574	1.763	1.649	1.96	1.762	1.438	2.32

- FFN 출력

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-0.401	-0.358	-0.51	0.452	-0.614	-0.568	0.004	-1.107
dim 1	-1.225	-1.108	-1.284	-2.287	-2.74	-1.282	-0.934	-1.704
dim 2	0.251	0.12	0.374	2.202	2.529	0.474	0.035	1.608
dim 3	4.093	4.191	4.11	2.044	3.857	4.164	3.905	3.264

2. Transformer Review: Decoder



- **Decoder FFN + 최종 출력:** FFN($4 \rightarrow 8 \rightarrow 4$, ReLU) $\rightarrow$ Residual + LayerNorm $\rightarrow$ 디코더 최종 출력

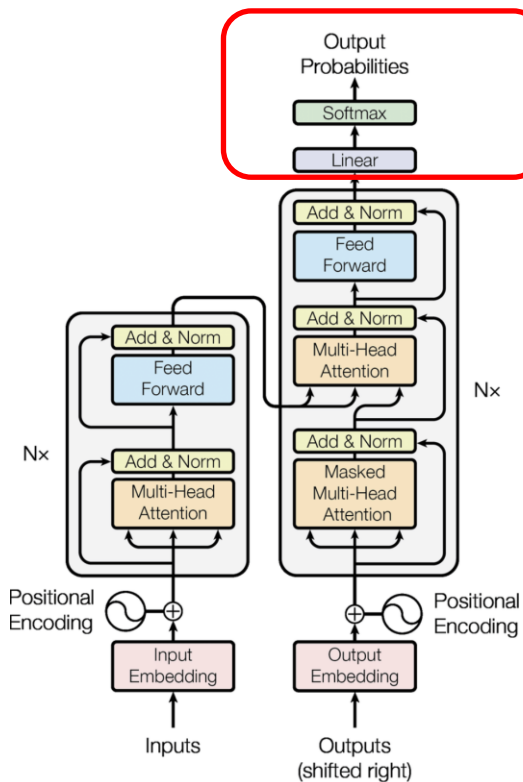
- +Residual

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1.582	-1.659	-1.713	0.867	-1.302	-1.868	-0.998	-1.739
dim 1	0.36	0.403	0.278	-2.268	-2.301	0.198	0.713	-0.157
dim 2	-0.027	-0.028	0.039	0.619	1.368	0.156	-0.092	0.522
dim 3	3.966	4.129	4.087	3.193	5.267	4.302	3.388	3.435

- LayerNorm $\rightarrow$ 디코더 최종 출력

차원	나는	오늘	공장	라인	에서	불량	을	발견
dim 0	-1.113	-1.119	-1.128	0.136	-0.703	-1.144	-1.069	-1.202
dim 1	-0.157	-0.146	-0.187	-1.481	-1.045	-0.223	-0.024	-0.359
dim 2	-0.348	-0.349	-0.3	0.008	0.208	-0.241	-0.516	0.004
dim 3	1.617	1.614	1.614	1.337	1.54	1.608	1.609	1.557

2. Transformer Review: Decoder



- **Linear Projection($d_{\text{model}} \rightarrow \text{Vocab_size}$)**

- Linear $\rightarrow$ Logit: Decoder 결과에 $W_{\text{proj}}(4 \rightarrow 12)$ 를 통해서 $\text{vocab_size}=12$ 개의 점수 계산

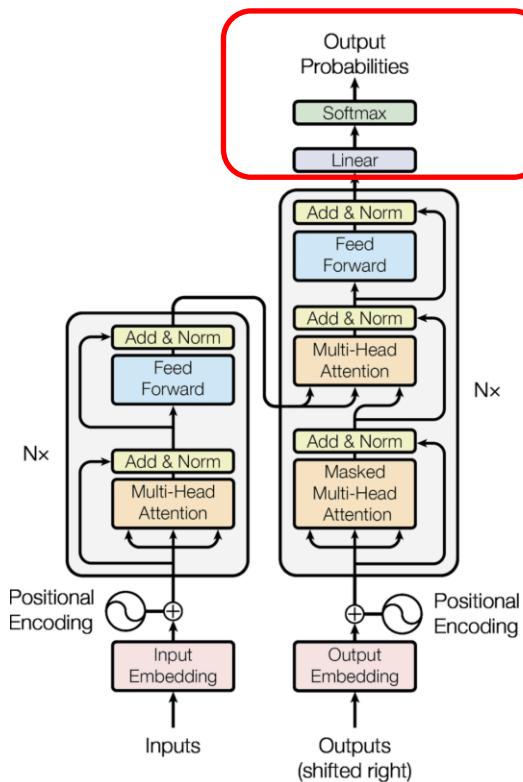
차원	나는	오늘	공장	라인	에서	불량	을	발견
나는	-0.104	-0.093	-0.114	-1.351	-0.804	-0.13	-0.032	-0.181
오늘	-0.259	-0.254	-0.255	-0.695	-0.452	-0.252	-0.256	-0.227
공장	1.339	1.35	1.367	-0.58	0.73	1.397	1.264	1.516
라인	-1.976	-1.986	-1.978	0.029	-1.14	-1.977	-1.983	-1.953
에서	-0.271	-0.261	-0.326	-1.2	-1.205	-0.395	-0.05	-0.666
불량	-1.474	-1.464	-1.54	-1.952	-2.349	-1.621	-1.206	-1.922
을	1.066	1.049	1.114	2.579	2.264	1.175	0.842	1.393
발견	-0.32	-0.331	-0.323	1.048	0.301	-0.323	-0.341	-0.331
했다	1.154	1.144	1.143	1.881	1.481	1.133	1.152	1.056
보고	-0.354	-0.363	-0.388	0.982	-0.022	-0.427	-0.261	-0.595
심각	-0.303	-0.306	-0.326	0.161	-0.289	-0.352	-0.231	-0.461
<EOS>	-2.414	-2.417	-2.421	-1.079	-1.968	-2.427	-2.379	-2.418

하이라이트: 각 위치에서 argmax선택된 vocab

- 컬럼: 각 위치(토큰)에서 12개 vocab에 대한 점수, "오늘" 토큰의 벡터($\text{dim}=4$)가 W_{proj} 곱해져서 vocab 12개에 대한 점수 12개로 변환, 즉, "오늘" 토큰 다음에 올 토큰 후보 12개의 점수이며, 그 값이 큰 점수의 토큰(형광색)을 선택 (Greedy)

`self.proj = nn.Linear(d_model, vocab_size)`

2. Transformer Review: Decoder



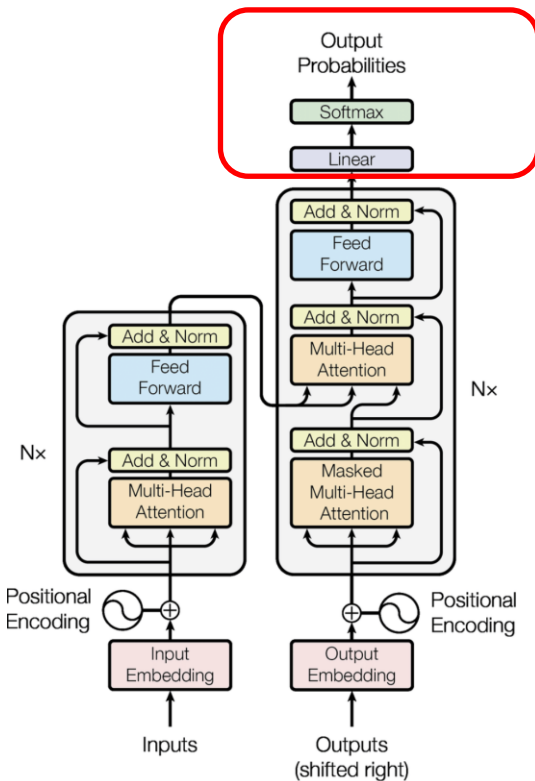
• **Softmax** → 확률 분포

• Logit → Softmax → 각 위치별 vocab 확률 (합=1)

vocab	나는	오늘	공장	라인	에서	불량	을	발견
나는	0.0603	0.0611	0.0592	0.0088	0.0212	0.0577	0.0667	0.0526
오늘	0.0516	0.052	0.0515	0.0169	0.0301	0.0511	0.0533	0.0502
공장	0.2552	0.2586	0.2605	0.0189	0.0983	0.2657	0.2437	0.2868
라인	0.0093	0.0092	0.0092	0.0348	0.0151	0.0091	0.0095	0.0089
에서	0.051	0.0516	0.0479	0.0102	0.0142	0.0443	0.0655	0.0324
불량	0.0153	0.0155	0.0142	0.0048	0.0045	0.013	0.0206	0.0092
을	0.1942	0.1914	0.2023	0.4458	0.4558	0.2128	0.1598	0.2536
발견	0.0486	0.0481	0.0481	0.0964	0.064	0.0476	0.049	0.0452
했다	0.2121	0.2105	0.2082	0.2218	0.2083	0.204	0.2179	0.1811
보고	0.047	0.0466	0.045	0.0903	0.0463	0.0429	0.053	0.0347
심각	0.0494	0.0494	0.0479	0.0397	0.0355	0.0462	0.0547	0.0397
	0.006	0.006	0.0059	0.0115	0.0066	0.0058	0.0064	0.0056

`probs = F.softmax(logits, dim=-1)`

2. Transformer Review: Decoder



• 생성 시 토큰 선택

- 첫 번째 위치 Top-3 후보 및 선택 방식 비교: 위치 0("나는") Top-3 후보

Top-k: 상위 k개 토큰 중
샘플링, 다양성

vocab	나는
나는	0.0603
오늘	0.0516
공장	0.2552
라인	0.0093
에서	0.051
불량	0.0153
을	0.1942
발견	0.0486
했다	0.2121
보고	0.047
심각	0.0494
	0.006

Greedy 선택: 항상 최고확률을 선택, 빠르고 결정적

Sampling 후보

Sampling 후보

Greedy

```
next = probs.argmax(dim=-1)
```

Top-k Sampling

```
top_k = torch.topk(probs, k=3)
```

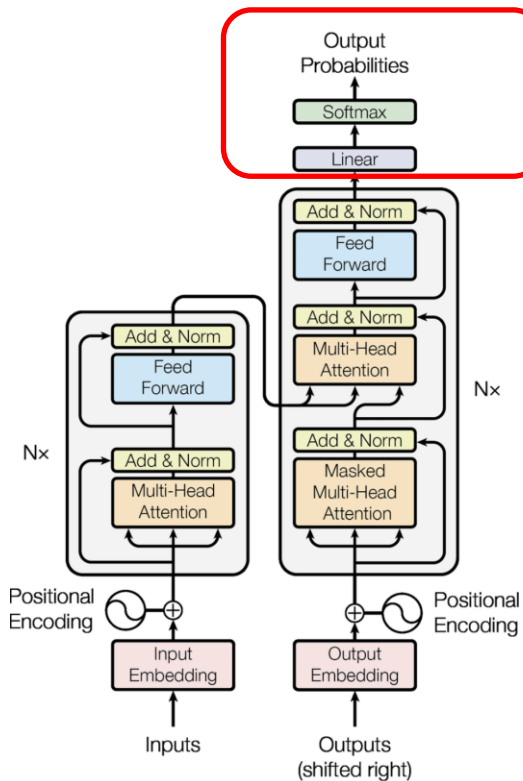
```
next = top_k.indices[torch.multinomial(top_k.values, 1)]
```

Temperature

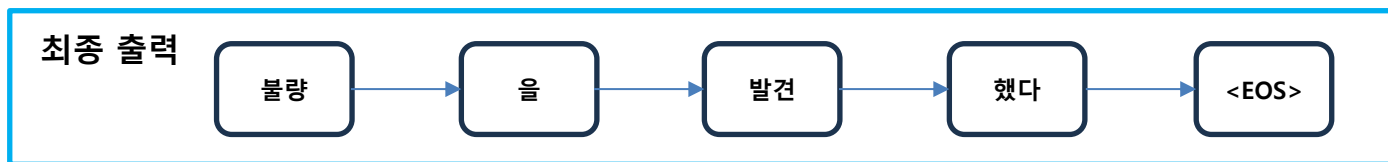
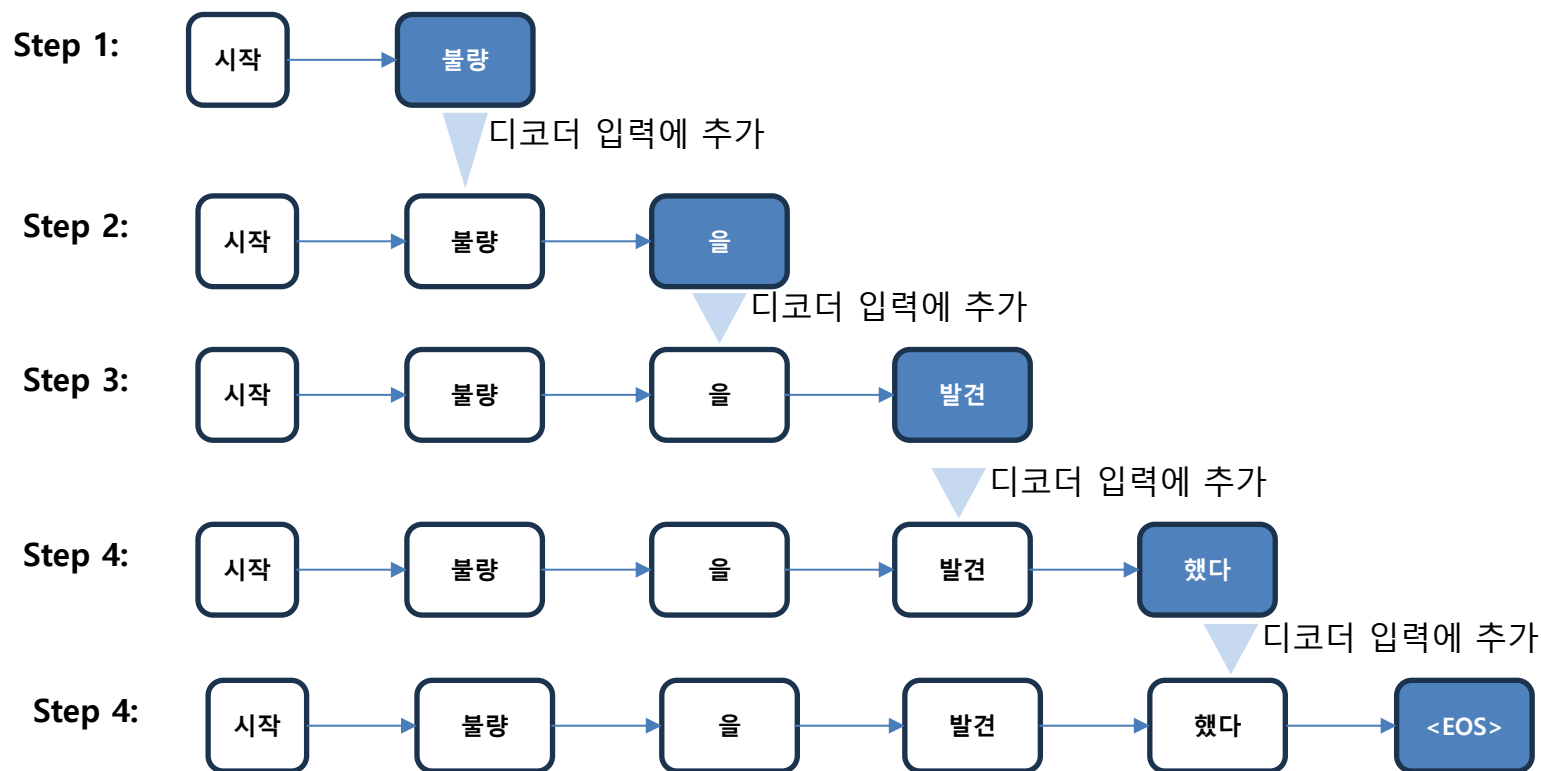
```
probs = F.softmax(logits / temperature, dim=-1)
```

Temperature값을 반영하여 낮으면 Sharp하게 하고, 창의성 조절을 할 수 있음.

2. Transformer Review: Decoder



- **자동 회귀 생성:** 생성된 토큰을 디코더입력에 추가하여 <EOS>가 나올 때까지 반복
- 학습 시: Teacher Forcing(정답시퀀스를 디코더입력으로 사용), 추론 시: 자동회귀(직전 생성 토큰을 디코더입력



```
while True:
    logits = model(enc_out, dec_input)
    next = logits[:, -1, :].argmax()
    if next == EOS: break
    dec_input = torch.cat([dec_input, next])
```

3. RAG 이해

- **LLM의 한계**

- Knowledge Cutoff: 특정 시점까지의 데이터로 학습
- Hallucination
- 개인화 불가

- **LLM 한계 극복을 위한 RAG**

- Retrieval-Augmented Generation
- Retrieval(검색): 관련 정보를 먼저 검색
- Augmented(증강): 그 정보로 LLM을 보강
- Generation(생성): 답변을 생성

3. RAG 이해

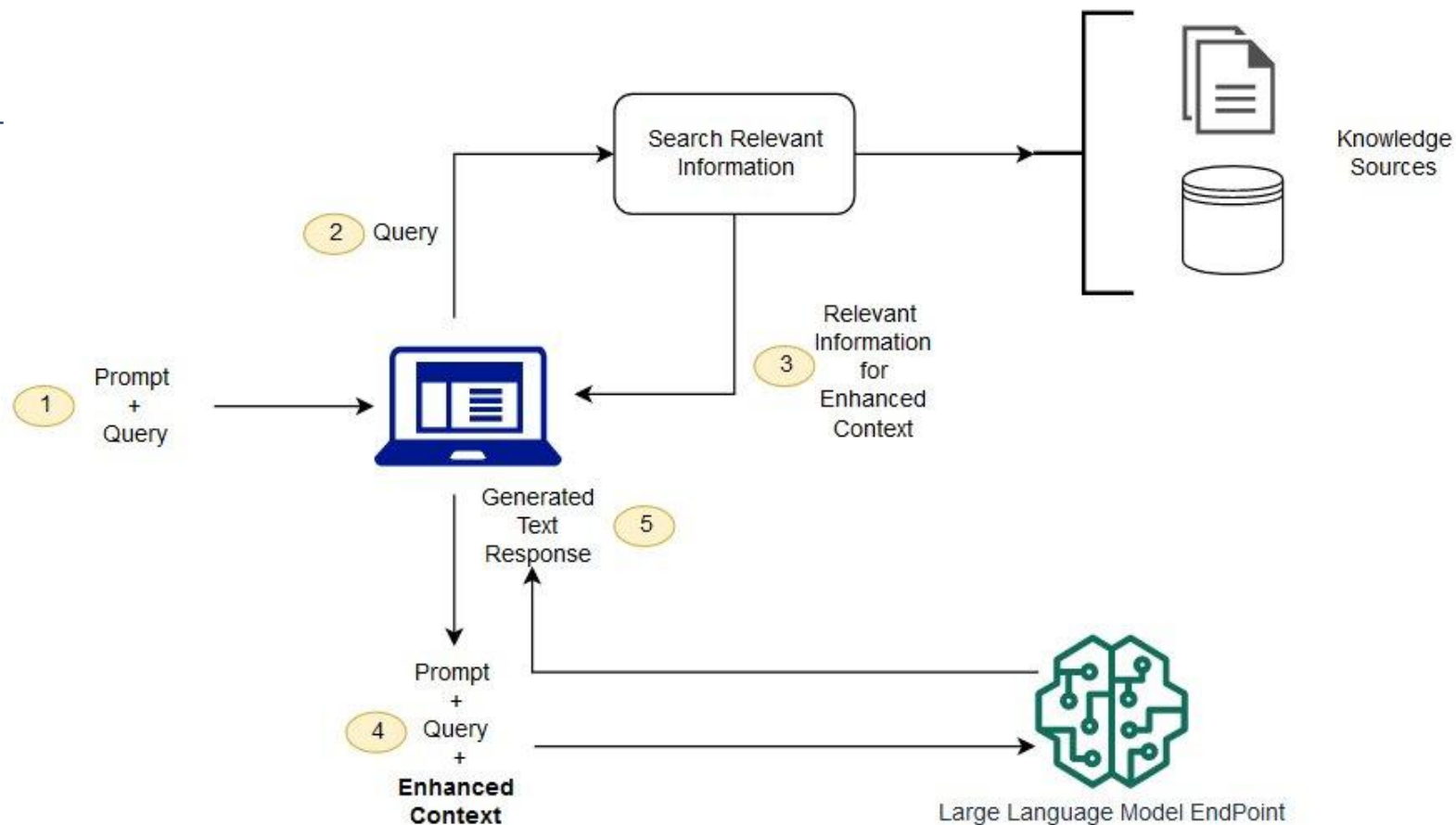
- **RAG란?**

- RAG(Retrieval-Augmented Generation): LLM이 답변 생성 전에 외부 문서를 검색해 prompt에 포함시키는 구조
- 동작 순서: 질의 임베딩 → 벡터DB에서 유사 문서 검색(retrieval) → 검색 결과를 prompt에 삽입 → context 기반 답변 생성
- Parametric memory(모델 가중치 내재 지식) + Non-parametric memory(외부 문서, 실시간 갱신 가능)의 결합 구조
- 장점: 재학습 없이 최신/도메인 지식 반영 가능, 출처 추적(citation) 가능
- 제조 적용 예: 설비 매뉴얼·SOP·불량 이력은 자주 갱신되고 기업마다 달라 fine-tuning보다 RAG가 적합한 경우가 많음

3. RAG 이해

- RAG 과정

- LLM 학습 데이터 외의 데이터인 외부 데이터 생성 후 임베딩을 통해 벡터 DB저장
- 질문에 대한 관련 정보 검색
- 외부 데이터 업데이트: 실시간 및 배치 방식



3. RAG 이해

- RAG: 검색 증강 생성
 - 1단계:
 - 사용자 질문 입력사용자가 질문과 함께 프롬프트 입력
 - 예: “2025년 전기차 보조금 정책은 어떻게 바뀌었는지 알려줘”
 - 2단계:
 - 질문 → 웹 검색 쿼리로 변환
 - 핵심 키워드 중심의 간결한 쿼리 생성예: “2025년 한국 전기차 보조금 변경 내용”
 - 3단계:
 - 웹 검색 및 정보 수집정부/언론 등 신뢰 가능한 출처 탐색
 - 주요 문장 추출 및 요약예: “보조금 600만 원 → 500만 원 축소, 고가 차량 제외”

3. RAG 이해

- RAG: 검색 증강 생성
 - 4단계:
 - 확장된 문맥 생성 (Enhanced Context)
 - 외부 정보 + 프롬프트 + 사용자 질문 결합 → LLM 입력용 고도화된 문맥 생성
 - 5단계:
 - LLM이 최종 응답 생성요약, 설명, 근거 등 사용자 맞춤 응답 출력
 - 예: "보조금이 500만 원으로 축소되며, 고가 차량은 제외..."

3. RAG 이해

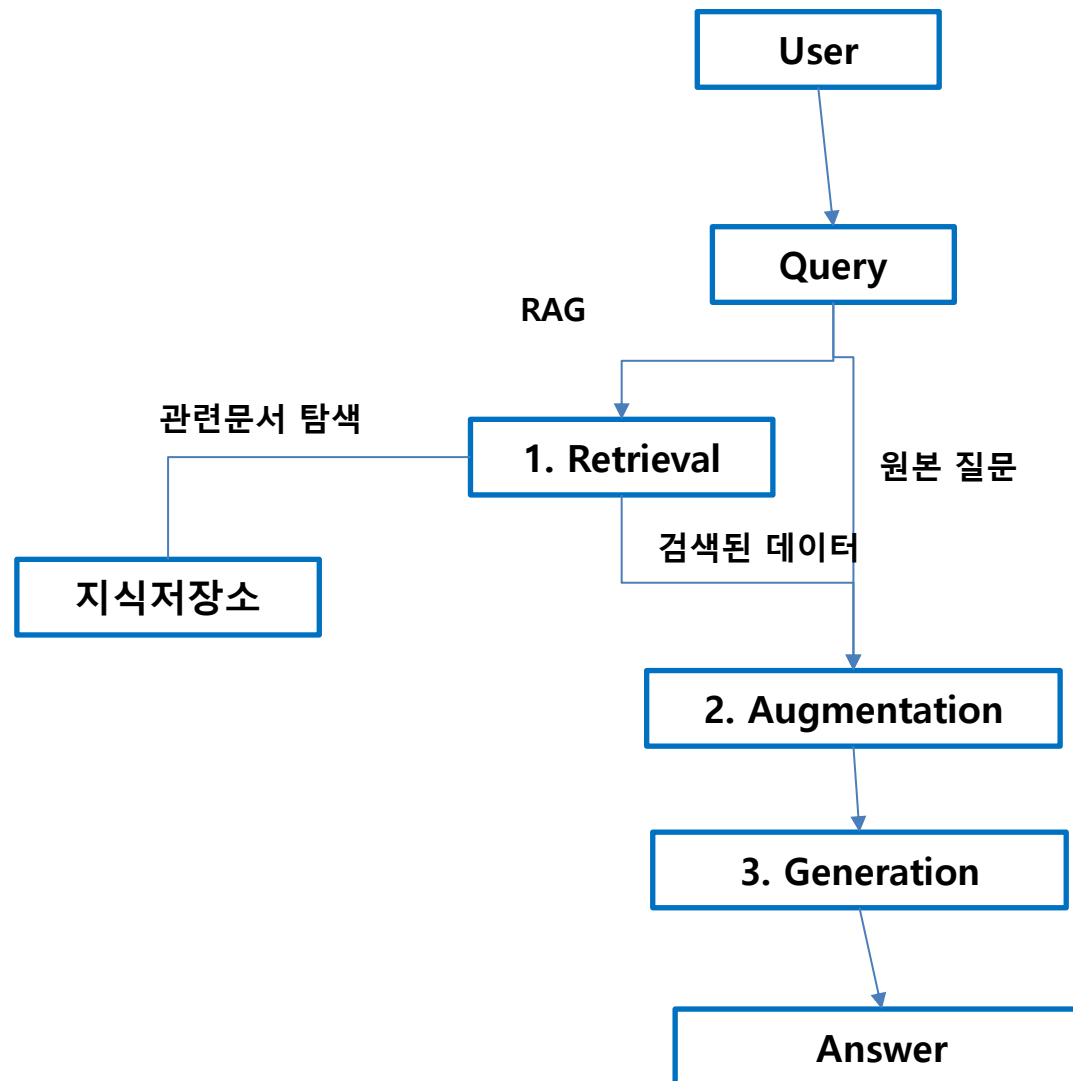
- **Fine Tuning**

- 기존 GPT 모델에 특정 데이터셋을 추가 학습시켜 도메인 특화 모델을 만드는 방법
- GPT의 기본 성능 + 맞춤형 지식/스타일 내재화
- Fine Tuning의 특징
 - 일회성 응답이 아닌 지속적으로 반영되는 학습 결과
 - 다음을 모델 자체에 자동 학습
 - 도메인 지식
 - 브랜드 어조
 - 업무 스타일 등

3. RAG 이해

- RAG의 구성요소

- 지식 저장소(Knowledge Base)
- 검색시스템(Retriever)
- 생성모델(Generator)



- **RAG의 주요 특성: Chunking**

- Chunking: 긴 문서를 검색 가능한 단위로 분할하는 과정 — chunk 크기가 검색 품질에 직접 영향
- Fixed-size: 고정 토큰/문자 수로 분할, 구현 단순하지만 문맥이 중간에 끊길 위험
- Semantic chunking: 의미 단위(문단, 주제 전환점)로 분할 — embedding 유사도 기반으로 경계 탐지
- Parent-Child chunking: 작은 단위(child)로 검색하고, 반환 시 더 큰 단위(parent)를 함께 줘서 문맥 보존
- Chunk overlap: 인접 chunk 간 일부 중첩(보통 10~20%)으로 경계 정보 손실 방지
- 제조 문서 특성: SOP는 절차 순서가 핵심 → chunking 시 단계가 중간에 끊기지 않도록 주의

3. RAG 이해

- **RAG의 주요 특성: 검색 정밀도**

- 검색 정밀도(retrieval precision) = 검색된 chunk 중 질의와 실제 관련된 chunk의 비율
- 정밀도가 낮으면 LLM이 무관한 context를 참고해 답변 품질 저하로 직결
- 영향 요인: embedding 모델 품질, chunk 크기/경계, Top-K 값(K가 너무 크면 잡음 증가)
- 평가 지표:
 - Recall@K, Precision@K, MRR(Mean Reciprocal Rank)
- 적용 예:
 - "베어링 이상" 질의 시 무관한 다른 설비 보고서가 함께 검색되는 상황을 개선

3. RAG 이해

- **RAG의 주요 특성: 환각**

- 환각(Hallucination): 사실에 근거하지 않은 내용을 그럴듯하게 생성하는 현상
- RAG에서도 발생 가능: ① context가 질의와 무관 ② context가 있어도 LLM이 무시하고 parametric 지식으로 답변 ③ context를 잘못 해석/조합
- Faithfulness(충실성): 답변이 제공된 context에 얼마나 근거하는지 측정하는 개념 (이후 RAGAS에서 논의)
- 완화 방안: "context에 없으면 모른다고 답하라" prompt 명시, 낮은 temperature, citation 강제
- 제조 현장에서의 리스크: 원인 진단을 잘못 환각하면 실제 조치 오류로 이어질 수 있어 특히 민감한 영역

3. RAG 이해

- **Advanced RAG: Hybrid Search(Dense+Sparse)**

- Dense retrieval: embedding 벡터 간 cosine similarity 기반 검색 — 의미적 유사성 포착에 강점
- Sparse retrieval: BM25/TF-IDF 등 키워드 매칭 기반 — 정확한 용어·숫자·고유명사 검색에 강점
- Dense의 약점: "사출성형기 #3호기" 같은 정확 일치가 필요한 질의에서 취약
- Hybrid Search: 두 점수를 가중합 또는 RRF(Reciprocal Rank Fusion)로 결합해 최종 순위 산출
- 제조 도메인에서 특히 유효할 수 있음
 - 예: 설비명·부품번호는 sparse, 증상 서술은 dense가 각각 담당

3. RAG 이해

- **Advanced RAG: Reranking (Cross-Encoder)**

- 1차 검색(Top-K retrieval)은 속도를 위해 단순 유사도로 후보를 빠르게 추출 (recall 위주)
- Cross-Encoder reranker: 질의와 각 후보 문서를 함께 인코딩해 더 정밀한 관련도 점수 계산 (precision 위주)
- Bi-encoder(임베딩 검색)와 차이: Bi-encoder는 질의/문서를 독립적으로 임베딩 후 비교, Cross-encoder는 쌍으로 함께 처리해 정확하지만 연산량 큼
- 구조: 1차로 Top-50~100개 빠르게 추출 → reranker로 재정렬 → 최종 Top-K(3~5)만 LLM에 전달
- 대표 모델: ms-marco-MiniLM, bge-reranker 등

3. RAG 이해

- **RAG to GraphRAG**

- RAG의 한계

- 점 검색

- 문서가 Chunking되어 질문과 가장 유사한 조각이 발견됨, 문서 내 정보간 연결고리 없음

- 맥락의 단절

- 문서가 Chunking되면 맥락이 사라짐

- Multi-hop 추론 불가

- 실제 질문에 여러 단계 추론이 필요할 수 있음
 - 기존 RAG에서는 처리하기 어려움

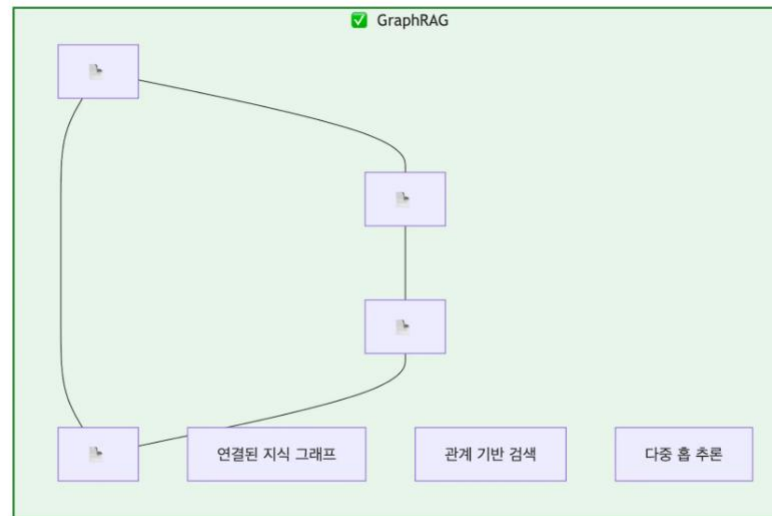
3. RAG 이해

- RAG to GraphRAG

- GraphRAG

- 지식 그래프

- 정보를 노드와 엣지로 표현
 - 전통 RAG의 벡터 검색에 지식 그래프 탐색을 결합



3. RAG 이해

- RAG to GraphRAG

- GraphRAG

- 지식 그래프

- 현실 세계의 지식을 그래프 형태로 표현

- 핵심 차이: 의미(Semantics)

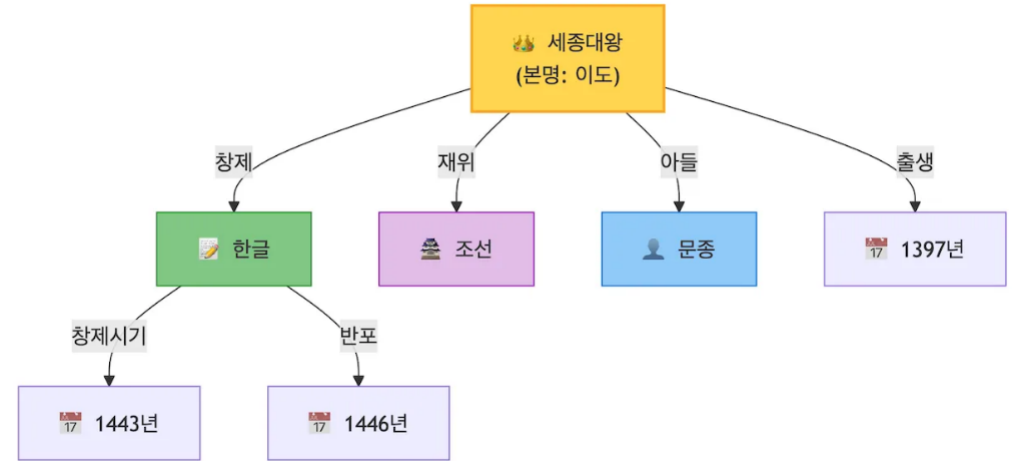
- 노드:개념, 사물

- 엣지: 관계

- 핵심 구성 요소

- 엔티티: 지식 그래프의 노드, 고유한 식별자, 여러 속성 가능, 다른 엔티티와 관계맺음

- 관계: 두 엔티티 사이의 연결을 의미있게 정의



3. RAG 이해

- **RAG to GraphRAG**

- GraphRAG

- 지식 그래프

- 속성: 엔티티, 관계 모두 갖을 수 있음

- 트리플 구조

- 지식 그래프의 가장 작은 단위
 - 주어-관계-목적어(Subject-Predicate-Object)

- 스키마

- 지식 그래프의 설계도
 - 어떤 종류의 엔티티와 관계 정의

4. LangChain 활용

- **LangChain이란**

- LangChain: LLM 애플리케이션 개발 프레임워크 — LLM·데이터·도구를 체인(Chain) 형태로 연결
- "LLM 호출"을 모듈화된 컴포넌트로 추상화해 재사용·조합 가능하게 설계
- 주요 구성요소: PromptTemplate, LLM Wrapper, Chain, Memory, Agent, ToolTF/Keras의 Sequential/Functional API처럼 "블록을 쌓는" 멘탈모델로 접근 가능
- 제조 적용: RAG 파이프라인, 멀티스텝 진단 워크플로우(증상 입력→원인 검색→조치 제안) 구성

4. LangChain 활용

- **LangChain 핵심 컴포넌트 이해**

- PromptTemplate: {context}, {question} 같은 placeholder로 동적 프롬프트를 생성하는 템플릿
- LLM Wrapper: OpenAI, Ollama, HuggingFace 등 다양한 LLM을 동일 인터페이스로 호출 가능하게 추상화
- Chain: 여러 컴포넌트를 순차 연결 (예: Retrieval → Prompt → LLM → Output Parser)
- Output Parser: LLM의 텍스트 출력을 구조화된 형태(JSON 등)로 변환
- Memory: 대화 이력을 유지해 멀티턴 대화 지원
- 코드 패턴: LCEL(LangChain Expression Language)로 prompt | llm | parser 형태로 체이닝

4. LangChain 활용

- **LangChain 핵심 컴포넌트 이해**

- PromptTemplate
- 동적 프롬프트 생성을 위한 템플릿 객체
- {context}, {question} 등 placeholder 를 정의하고, 실행 시 실제 값으로 치환
- 하드코딩된 문자열 대신 재사용 가능한 프롬프트 구조 확보
- ChatPromptTemplate, HumanMessagePromptTemplate 등 대화형 변형도 제공
- 예시: `PromptTemplate(template="질문: {question}\n답변:", input_variables=["question"])`

4. LangChain 활용

- **LangChain 핵심 컴포넌트 이해**

- LLM Wrapper
- OpenAI, Ollama, HuggingFace 등 이기종 LLM을 단일 인터페이스로 추상화
- 모델 교체 시 코드 변경 최소화 → 벤더 종속성(vendor lock-in) 탈피
- ChatOpenAI, OllamaLLM, HuggingFacePipeline 등 클래스로 구분
- .invoke(), .stream() 등 공통 메서드로 호출
- 로컬 모델 ↔ 클라우드 API 전환도 wrapper 교체만으로 처리

4. LangChain 활용

- **LangChain 핵심 컴포넌트 이해**

- Chain
- 여러 컴포넌트를 순서대로 연결하는 실행 파이프라인
- 대표 흐름: Retrieval → Prompt → LLM → Output Parser
- 각 단계의 출력이 다음 단계의 입력으로 자동 전달
- LLMChain, RetrievalQA, ConversationalRetrievalChain 등 목적별 존재
- LCEL 방식(| 연산자)으로 선언적·간결한 체인 구성 가능

4. LangChain 활용

- **LangChain 핵심 컴포넌트 이해**

- Output Parser
- LLM이 반환하는 비정형 텍스트를 구조화된 데이터로 변환
- 주요 파서 유형:
 - StrOutputParser — 문자열 그대로 반환
 - JsonOutputParser — JSON 객체로 파싱
 - PydanticOutputParser — Pydantic 모델로 검증·변환
- 파서가 LLM에게 출력 형식 지침(format instructions) 을 자동 삽입
- 다운스트림 코드에서 .key 접근 등 프로그래밍 방식 활용 가능

4. LangChain 활용

- **LangChain 핵심 컴포넌트 이해**

- Memory
- 대화 이력(history) 을 저장·관리하여 멀티턴 대화 구현
- 주요 Memory 유형:
 - ConversationBufferMemory — 전체 이력 보존
 - ConversationSummaryMemory — LLM이 이력을 요약하여 압축
 - ConversationWindowMemory — 최근 N턴만 유지
- 이력은 프롬프트에 자동 삽입되어 문맥 유지장치 대화 시 토큰 비용 관리 전략과 함께 설계 필요

4. LangChain 활용

- LangChain 핵심 컴포넌트 이해

- 코드 패턴
- 컴포넌트를 | 파이프 연산자로 연결하는 선언적 체이닝 문법
- 기본 패턴: prompt | llm | parser
- 장점:
 - 코드가 데이터 흐름 그대로 읽혀 가독성 우수.
 - stream(), .batch(), .ainvoke() 등 실행 모드 일괄 지원
 - 중간 단계 교체·삽입이 용이해 모듈성 확보
- 예시:

```
python chain = prompt | llm | StrOutputParser()  
  
chain.invoke({"question": "RAG란?"})
```

4. LangChain 활용

- **임베딩 모델 비교: BGE / E5 / Ko-sroberta**

- BGE(BAAI General Embedding): 다국어/영어 검색 성능 우수, MTEB 벤치마크 상위권
- E5: "query: "/"passage: " prefix를 붙여 사용하는 방식이 특징, contrastive learning 기반
- Ko-sroberta: 한국어 특화 sentence embedding, 한국어 의미 유사도 task에 강점
- 비교 기준: 임베딩 차원 수, 한국어 지원 수준, 검색 속도, 벤치마크 점수(MTEB/KLUE)

4. LangChain 활용

- QA Chain이란

- QA Chain: 질문 입력 → 관련 문서 검색 → 검색 결과 기반 답변 생성으로 이어지는 체인
- RetrievalQA 패턴: Retriever(벡터DB 검색기) + LLM을 결합한 가장 기본적인 RAG 체인 구조
- 체인 단계: 질문 임베딩 → 검색 → context 결합 → prompt 구성 → LLM 호출 → 답변 반환
- Stuff / Map-Reduce / Refine: 여러 문서를 LLM에 전달하는 전략 차이 (한번에 넣기 / 나눠 처리 후 합치기 / 순차 개선)
- 제조 적용 예: "설비 진동 이상의 원인은?" → 관련 보고서 검색 → 종합 답변 생성

4. LangChain 활용

- **QA Chain**이란

- QA Chain
- 질문(Question) → 검색(Retrieval) → 답변(Answer) 의 3단계로 구성된 파이프라인
- 단순 LLM 호출과의 차이:
 - 외부 문서를 실시간으로 참조하여 답변 근거 확보
 - 모델이 학습하지 않은 도메인 특화 지식도 검색을 통해 활용 가능
 - RAG(Retrieval-Augmented Generation)의 가장 기본적인 구현 형태
- 핵심 가치: 환각(Hallucination) 억제 + 출처 추적 가능

4. LangChain 활용

- RetrievalQA 패턴

- LangChain에서 RAG를 구현하는 가장 기본적인 체인 클래스

구성 요소	역할
Retriever	벡터DB에서 관련 문서 청크 검색
LLM	검색 결과를 바탕으로 답변 생성
Prompt	검색 문서 + 질문을 LLM에 전달하는 템플릿

- 생성 예시: chain_type 파라미터로 문서 처리 전략 선택 가능 (stuff, map_reduce, refine)

```
chain = RetrievalQA.from_chain_type(  
    llm=llm,  
    retriever=vectorstore.as_retriever()  
)
```

4. LangChain 활용

- 체인 실행 단계 (Step-by-Step)

- 각 단계는 LCEL()로 연결되어 자동 순차 실행
- 검색 단계에서 k 값(반환 문서 수) 조정이 품질에 큰 영향

사용자 질문

↓

① 질문 임베딩 질문 텍스트 → 벡터로 변환

↓

② 유사도 검색 벡터DB에서 상위 k개 문서 청크 반환

↓

③ context 결합 검색된 문서들을 하나의 텍스트 블록으로 합산

↓

④ prompt 구성 PromptTemplate에 {context} + {question} 삽입

↓

⑤ LLM 호출 완성된 프롬프트를 LLM에 전달

↓

⑥ 답변 반환 Output Parser를 통해 최종 답변 출력

- 문서 처리 전략: Stuff / Map-Reduce / Refine

- ① Stuff (기본값) 검색된 모든 문서를 한 번에 프롬프트에 삽입 후 LLM 호출
 - 장점: 구현 단순, 빠름 / 단점: 문서가 많으면 컨텍스트 길이 초과 위험
- ② Map-Reduce 각 문서를 개별적으로 LLM에 처리(Map) → 결과를 합쳐 최종 답변(Reduce)
 - 장점: 대용량 문서 처리 가능 / 단점: LLM 호출 횟수 증가 → 비용·속도 불리
- ③ Refine 첫 문서로 초안 답변 생성 → 다음 문서를 보며 순차적으로 답변을 개선
 - 장점: 문서 간 맥락 누적 가능 / 단점: 순차 처리라 속도 가장 느림

4. LangChain 활용

- **제조 현장 적용 예시: “설비 진동 이상의 원인은 무엇인가?”**
 - 도메인 문서(보고서, 매뉴얼, 이력서) 를 벡터DB에 사전 적재하는 것이 핵심
 - 답변에 출처 문서 표시 기능 추가 시 현장 신뢰도 향상
 - 확장: 품질 불량 원인 분석 / 설비 교체 주기 판단 / 안전 규정 조회

- ① 질문 임베딩
"설비 진동 이상 원인" → 벡터 변환
- ② 벡터DB 검색
 - 베어링 마모 점검 보고서 (2024-03)
 - 불균형 진동 분석 매뉴얼
 - 이상 진동 조치 이력서 (설비 #A-07)
- ③ context 결합 + prompt 구성
[검색 문서 3건] + "원인 분석 및 조치 방안을 설명하라"
- ④ LLM 답변 생성
"베어링 마모 및 회전체 불균형이 주요 원인으로 추정되며,
2024년 3월 보고서 기준 A-07 설비의 교체 주기 도래..."

4. LangChain 활용

- **Embedding Similarity 비교**

- 유사도 계산 방식:

- Cosine Similarity(방향 기반, 가장 일반적), Dot Product(크기+방향), Euclidean Distance(거리 기반)

- Cosine similarity 공식: $\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$ — 방향이 같으면 1, 직각이면 0

- 임베딩이 정규화(normalize)되어 있으면 dot product와 cosine이 동일해지는 경우 있음

- 예:

- "베어링 마모"/"베어링 손상"처럼 표현은 다르지만 의미가 같은 문장 쌍과, 의미가 다른 문장 쌍의 유사도를 비교해 임베딩 품질을 체감

4. LangChain 활용

- **ChromaDB**

- 오픈소스 벡터 데이터베이스, 로컬/임베디드 환경에서 가볍게 실행 — Colab 실습에 적합
- 핵심 기능: 문서+임베딩 저장, metadata 필터링, 유사도 검색(query) 지원
- Collection 단위로 문서 그룹 관리 — 제조 도메인에서는 설비별/부서별 컬렉션 분리 가능
- 영속성(persistence): 디스크 저장으로 세션 간 재사용 가능(PersistentClient)
- LangChain 연동: Chroma.from_documents()로 손쉽게 vector store 생성

4. LangChain 활용

- **ChromaDB**

- 오픈소스 벡터 데이터베이스 — AI/LLM 애플리케이션에 최적화된 경량 벡터 저장소
- 로컬 및 임베디드 환경에서 별도 서버 없이 실행 가능
- Colab, Jupyter 등 노트북 환경에서 즉시 사용 가능 → 실습·프로토타이핑에 적합
- `pip install chromadb`

	ChromaDB	Pinecone	FAISS
설치 방식	로컬/임베디드	클라우드 SaaS	로컬 라이브러리
서버 필요	X	✓	X
metadata 필터	✓	✓	X
영속성	✓	✓	별도 구현

4. LangChain 활용

• ChromaDB 기능

- ① 문서 + 임베딩 저장
 - 텍스트, 임베딩 벡터, metadata를 함께 저장하는 통합 구조
 - 임베딩 함수 지정 시 텍스트 입력만으로 자동 벡터 변환·저장
- ② Collection 단위 관리
 - 문서 그룹을 Collection으로 분리 관리
 - 제조 도메인 적용 예:설비_A07_보고서 / 품질_불량_이력 / 안전_규정_매뉴얼 등 목적별 컬렉션 분리
 - 설비별·부서별 독립 검색 공간 구성 가능
- ③ metadata 필터링
 - 유사도 검색 시 metadata 조건을 추가하여 검색 범위 제한

	ChromaDB	Pinecone	FAISS
설치 방식	로컬/임베디드	클라우드 SaaS	로컬 라이브러리
서버 필요	X	✓	X
metadata 필터	✓	✓	X
영속성	✓	✓	별도 구현

4. LangChain 활용

- ChromaDB 기능

- ④ 영속성 (PersistentClient)
 - 디스크에 저장 → 세션 종료 후에도 데이터 유지
 - 반복 실습 시 임베딩 재계산 불필요 → 시간·비용 절감

```
client = chromadb.PersistentClient(path="./chroma_db")
```

4. LangChain 활용

- ChromaDB 기능

- LangChain 연동: Chroma.from_documents() 한 줄로 문서 → 임베딩 → 벡터DB 저장 완료
- 기존 DB 재사용 시: Chroma(persist_directory=..., embedding_function=...) 로 로드만 하면 됨

```
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# 문서 로드 & 분할 (생략)
# docs = text_splitter.split_documents(raw_docs)

vectorstore = Chroma.from_documents(
    documents=docs,                # 분할된 문서 청크
    embedding=OpenAIEmbeddings(),  # 임베딩 모델
    persist_directory="./chroma_db", # 영속성 경로
    collection_name="설비_매뉴얼"   # 컬렉션 이름
)

# Retriever로 변환 → QA Chain에 바로 연결
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```


4. LangChain 활용

- ChromaDB 기능

- 전체 RAG 파이프라인에서의 위치:

```
문서 적재
↓
Chroma.from_documents() ← 여기서 ChromaDB 생성
↓
vectorstore.as_retriever() ← Retriever 객체로 변환
↓
RetrievalQA Chain 연결 ← QA Chain에 전달
↓
사용자 질문 → 검색 → 답변
```

- **FAISS Indexing**

- Facebook AI Similarity Search — 대규모 벡터 검색에 최적화된 인메모리 기반 라이브러리, 매우 빠름
- 인덱스 종류: Flat(완전탐색, 정확하지만 느림), IVF(역색인 기반 근사), HNSW(그래프 기반 근사) — 속도/정확도 trade-off
- ChromaDB와 차이: ChromaDB는 메타데이터·영속성 등 DB 기능 포함, FAISS는 순수 검색 성능에 특화
- 대규모 문서셋(수만~수백만 건)에서는 근사 인덱스(IVF/HNSW)가 필요
- 제조 적용 시점: 소규모 PoC는 ChromaDB로 충분, 다공장/대량 이력 데이터로 확장 시 FAISS 고려

4. LangChain 활용

- **FAISS Indexing**

- Facebook AI Similarity Search: Meta(구 Facebook) AI Research에서 개발한 오픈소스 벡터 검색 라이브러리
- 특징
 - 인메모리(In-memory) 기반 — 디스크 I/O 없이 RAM에서 직접 연산
 - 수백만~수십억 개의 고차원 벡터를 밀리초 단위로 검색
 - GPU 가속 지원 (CUDA)
 - 순수 검색 성능에만 집중한 경량 라이브러리

문서들 → 임베딩 모델 → 벡터 변환 → FAISS 인덱스에 저장

질문 → 임베딩 모델 → 질문 벡터 → 인덱스에서 가장 가까운 벡터 k개 반환

```
import faiss
```

```
# 정규화된 임베딩 → Inner Product = Cosine Similarity
```

```
faiss_index = faiss.IndexFlatIP(dim) # dim=768
```

```
faiss_index.add(doc_embeddings) # 60건 추가
```

4. LangChain 활용

- 인덱스 종류와 속도/정확도 Trade-off

- Flat (완전 탐색)

- 모든 벡터와 일일이 거리 계산 / 정확하지만 건수가 늘수록 선형적으로 느려짐/소규모에서 사용

- IVF (Inverted File Index)

- 벡터 공간을 클러스터로 나눠 후보 클러스터만 탐색
 - 탐색 범위를 좁혀 속도를 높이되 약간의 정확도 손실

- HNSW (Hierarchical Navigable Small World)

- 계층적 그래프 구조로 가까운 벡터끼리 연결
 - 현재 가장 널리 쓰이는 근사 탐색 방식
 - ChromaDB 내부도 기본적으로 HNSW 사용

인덱스	방식	속도	정확도	적합한 규모
IndexFlatIP/L2	완전 탐색	느림	100% (정확)	~수만 건
IndexIVF	역색인 기반 근사	빠름	95~99%	수십만~수백만 건
IndexHNSW	그래프 기반 근사	매우 빠름	95~99%	수백만 건 이상

4. LangChain 활용

- FAISS VS Chroma DB:소규모에서는 차이가 미미하고, 건수가 커질수록 FAISS의 속도 우위

항목	FAISS	ChromaDB
목적	순수 벡터 검색 성능	RAG용 벡터 DB
저장 방식	인메모리 (기본)	디스크 영속성 지원
메타데이터 필터링	직접 지원 안 함	where={"심각도":"상"}
검색 속도	매우 빠름	빠름 (HNSW 기반)
설치/사용 난이도	낮음	낮음
LangChain 연동	가능	기본 지원
재시작 후 유지	(별도 저장 필요)	

4. LangChain 활용

• 제조 활용

- 검색 속도보다 메타데이터 필터링이 더 중요하다면 → ChromaDB
- 문서 수가 수만 건을 넘고 latency가 문제되면 → FAISS 전환 검토
- 두 가지를 병행할 경우 → FAISS로 후보 추출 후 ChromaDB 메타데이터로 재필터링

소규모 PoC / 단일 공장

문서 수 ~수천 건

→ ChromaDB

이유: 메타데이터 필터링, 영속성, LangChain 연동이 쉬움

예: 특정 라인의 불량 보고서 검색 챗봇

중규모 / 다공장 통합

문서 수 수만~수십만 건

→ ChromaDB + HNSW 튜닝 or FAISS IVF

이유: 검색 latency가 체감되기 시작하는 구간

예: 전국 공장 설비 이력 통합 검색

대규모 / 실시간 생산 이력

문서 수 수백만 건 이상

→ FAISS IVF + GPU 가속

이유: 응답속도 SLA(ex. 200ms 이내) 달성 필요

예: 실시간 공정 이상 감지 + 유사 사례 즉시 검색

5. Ollama 이해 및 실습

- Ollama 개요

- 로컬 환경에서 LLM을 손쉽게 실행/관리할 수 있는 오픈소스 런타임
- GGUF(GPT-Generated Unified Format) 기반 — 양자화된 모델을 CPU/GPU에서 효율적으로 구동
- 클라우드 API 대비 장점: 데이터가 외부로 나가지 않음(온프레미스 보안), API 비용 없음, 네트워크 의존성 없음
- 단점: 로컬 하드웨어 성능에 따른 속도/모델 크기 제약, 최신 최상위 모델 대비 성능 한계
- 제조 현장 적합성: 설비 데이터·불량 정보 등 민감 데이터를 외부로 보내지 않아야 하는 보안 요구사항에 부합

5. Ollama 이해 및 실습

- Ollama 개요

- 로컬 환경에서 LLM을 손쉽게 실행·관리하는 오픈소스 런타임
 - 기존에 LLM을 로컬에서 돌리려면 모델 다운로드, 양자화, 의존성 설치 등 복잡한 과정이 필요
 - Ollama는 이 모든 것을 ollama run 모델명 한 줄로 해결

- GGUF 포맷

- GPT-Generated Unified Format
- 대형 모델을 4bit/8bit로 양자화(Quantization) 하여 용량과 메모리를 대폭 축소
- 원본 70B 모델 → 4bit 양자화 시 약 40GB → 일반 워크스테이션에서 구동 가능
- CPU 전용 실행도 가능 (GPU 없어도 동작, 단 속도 차이 있음)

```
# Ollama 설치
!curl -fsSL https://ollama.ai/install.sh | sh

# 백그라운드 서버 실행
!ollama serve &

# 모델 다운로드 및 실행
!ollama pull llama3          # 약 4.7GB
!ollama pull gemma3:4b      # 경량 모델
!ollama pull EEVE-Korean-10.8B # 한국어 특화
```


5. Ollama 이해 및 실습

• 클라우드 API VS Ollama

항목	클라우드 API (GPT-4, Claude)	Ollama (로컬)
데이터 보안	외부 서버로 전송	로컬에서만 처리
비용	토큰당 과금	무료
네트워크	인터넷 필수	오프라인 가능
모델 성능	최상위 (GPT-4o 등)	상대적으로 낮음
속도	빠름 (대형 인프라)	하드웨어 의존
최신 모델	즉시 사용 가능	공개 모델만 가능
설정 난이도	API 키만 필요	설치·관리 필요

```
from langchain_community.llms import Ollama

llm = Ollama(model="llama3", temperature=0.3)

# RetrievalQA에 그대로 교체
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,          # kogpt2 → Ollama로 교체
    retriever=retriever,
    return_source_documents=True,
)
```

5. Ollama 이해 및 실습

- 제조 현장 적합성

- 제조 현장의 데이터는 대부분 외부 유출이 불가한 민감 정보
- 클라우드 API 사용 시 질문 내용(= 설비 데이터 + 불량 정보)이 외부 서버로 전송: 기업 보안 정책, ISO 27001, 개인정보보호법 등 저촉 가능
- Ollama 사용 시 공장내부 폐쇄망 내 Chroma DB(설비이력 등) + Retriever + Ollama LLM(로컬 실행)을 통한 답변 생성

5. Ollama 이해 및 실습

- **Ollama 설치 및 로컬 LLM (Llama3/Mistral/Gemma) 실행**

- 설치: `curl -fsSL https://ollama.com/install.sh | sh` (Linux/Colab) 또는 공식 설치파일(Windows/Mac)
- 모델 실행: `ollama run llama3` 형태로 모델명만 지정하면 자동 다운로드+실행
- 모델 비교: Llama3(Meta, 범용 균형형), Mistral(경량, 빠른 추론), Gemma(Google, 경량+안전성 강조)
- 양자화(quantization) 옵션: 4bit/8bit 등 — 모델 크기 vs 정확도 trade-off, 태그로 선택(예: llama3:8b-instruct-q4_0)
- 예: 동일 제조 질문을 세 모델에 던져 응답 속도/품질/스타일 차이 비교

5. Ollama 이해 및 실습

• Modelfile 작성 및 시스템 프롬프트 커스텀

- Modelfile: Ollama에서 모델 동작을 정의하는 설정 파일 (Dockerfile과 유사한 개념)
- 주요 지시어: FROM(베이스 모델), SYSTEM(시스템 프롬프트), PARAMETER(temperature 등), TEMPLATE(prompt 포맷)
- 시스템 프롬프트로 역할 고정: "당신은 설비 진단 전문가입니다. 제공된 정보 외 추측은 하지 마세요" 형태로 삽입
- 빌드: ollama create my-model -f Modelfile → ollama run my-model로 호출 가능한 커스텀 모델 등록
- 예: 파인튜닝 모델을 GGUF로 변환한 뒤에도 동일한 방식으로 Modelfile 작성·등록

```
# Modelfile
# 1. 베이스 모델 지정
FROM llama3

# 2. 시스템 프롬프트 — 역할 고정
SYSTEM """
당신은 제조 현장 설비 진단 전문가입니다.
역할:
- CNC 머신, 프레스, 용접 설비 등의 이상 징후를 분석합니다.
- 제공된 불량 보고서와 설비 이력 데이터만을 근거로 답변합니다.
- 데이터에 없는 내용은 추측하지 말고 "제공된 정보만으로는 판단이 어렵습니다"라고
  답하세요.
- 답변은 항상 한국어로, 간결하고 명확하게 작성하세요.
- 원인 분석 시 4M(Man/Machine/Material/Method) 분류를 기준으로 서술하세요.
"""

# 3. 생성 파라미터
PARAMETER temperature 0.1      # 낮을수록 일관된 답변 (진단용이므로 낮게 설정)
PARAMETER top_p 0.9
PARAMETER num_ctx 4096         # 컨텍스트 윈도우 (RAG 문서 여러 건 담을 크기)

# 4. 프롬프트 템플릿
TEMPLATE """
{{ if .System }}<|system|>
{{ .System }}<|end|>
{{ end }}
<|user|>
{{ .Prompt }}<|end|>
<|assistant|>
"""
```

5. Ollama 이해 및 실습

- **제조 QA 챗봇 프롬프트 설계**

- 설계 원칙: 역할 정의(누구인지) + 제약사항(모르면 모른다고 답하기) + 출력 형식 지정
- Few-shot 예시: 질문-답변 예시 1~2개를 시스템 프롬프트에 포함해 응답 스타일 고정
- 도메인 용어집 주입: 설비명·약어·단위를 명시해 일관된 용어 사용 유도
- 환각 방지 문구: "제공된 context에 없는 내용은 추측하지 말고 모른다고 답하라" 명시
- 예: 원칙 적용 전/후 프롬프트로 동일 질문에 대한 응답 품질 비교

5. Ollama 이해 및 실습

- **Ollama + LangChain 연동**

- LangChain의 ChatOllama(또는 Ollama) Wrapper로 로컬 모델을 체인에 연결
- 연동 구조: 로컬 Ollama 서버(기본 11434 포트) ↔ LangChain 클라이언트가 REST API로 통신
- 예: 앞서 만든 QA Chain의 LLM 백엔드를 HF/클라우드 모델에서 Ollama로 교체
- 장점: Chain 코드 구조는 그대로 유지한 채 백엔드만 교체 — LangChain 추상화의 핵심 효과 확인
- 다음 단계 예고: 이후 이 체인에 그래프RAG, 파인튜닝 모델이 차례로 결합됨

Industrial Data Science Lab

Contact:

won.sang.l@kangwon.ac.kr

<https://sites.google.com/view/idslab>